

R.E.D.

Real-time Environment Driver

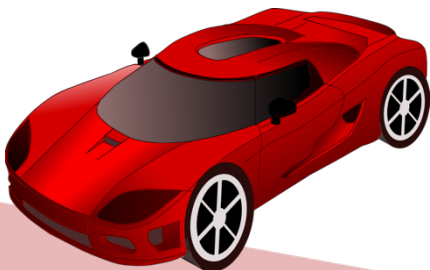
Unsere Vision ist es, ein interaktives Fahrerlebnis zu schaffen, bei dem Nutzer ein Fahrzeug sowohl manuell steuern als auch autonom navigieren lassen können. Durch R.E.D. wird eine mobile Applikation mit einem sensorgestützten Fahrzeug kombiniert, das flexibel auf seine Umgebung reagiert und verschiedene Fahrmodi ermöglicht. Das Projekt stellt ein praxisnahes Beispiel im Bereich mobiler Anwendungen, Echtzeitkommunikation und eingebetteter Systeme dar.

Autoren: Sebastian Fuchs, Alexa van der Meulen, Jonathan Schmidt, Vera Schwarz

Kurs: TINF24CS2

Hochschule: DHBW Mannheim

Datum: Juni 2026



Inhaltsverzeichnis

Einleitung.....	3
Vorgehensweise	4
User Stories.....	6
User Cards.....	9
Anforderungsanalyse, abgeleitet aus den User Stories	12
Vereinbarungen zur Umsetzung	13
Verbindung der Website oder Applikation mit dem Auto	14
Fernsteuerung	14
Hardware	14
Funktionen für die Überprüfung	14
Raum Scannen	14
Entwurf.....	15
Entwurf des Benutzerinterfaces	15
Willkommenseite	16
Modusauswahl.....	17
Manueller Fahrmodus	17
Routenzeichenmodus.....	18
Autonomer Modus	19
Entwurf der Hardwarekomponenten am Fahrzeug	19
Architektur.....	21
Softwareschichten	22
Architektur des Frontends	23
Schichtmodell des Frontends.....	23
Grundstruktur des Frontends	24
Darstellung der unterschiedlichen Seiten.....	26
AutonomDrivingPage	29
DrawingPage	31
Kommunikationsintegration.....	32
Befehlssystem	34
Struktur des Backends und der Hardware.....	36

Verwendete Technologien: Übersicht	36
Umsetzung mit C++ und dem Arduino Framework	36
Systemarchitektur	37
Hardware Spezifikationen	38
Software-Komponenten	40
Tests und Qualitätssicherung	47
Reflexion / Fazit.....	51

Einleitung

Dieses Dokument stellt die umfassende Dokumentation des Software-Engineering-Projekts R.E.D. – Real-time Environment Driver dar, das im Rahmen des Kurses TINF24CS2 an der DHBW Mannheim durchgeführt wurde. Ziel des Projekts ist die Entwicklung eines interaktiven Fahrerlebnisses, bei dem ein ferngesteuertes Fahrzeug über eine mobile Applikation sowohl manuell gesteuert als auch autonom betrieben werden kann. Dabei wird ein kleines, sensorgestütztes Fahrzeug eingesetzt, das mithilfe eines Abstandssensors seine Umgebung wahrnimmt und auf Hindernisse reagiert.

Das Dokument beschreibt sowohl die Projektergebnisse als auch das methodische Vorgehen der Projektgruppe. Dabei wird der gesamte Entwicklungsprozess von der Anforderungsanalyse über den Entwurf bis hin zur technischen Umsetzung strukturiert dargestellt.

Die Dokumentation umfasst insbesondere:

- User Stories und User Cards als Grundlage der funktionalen Anforderungen
- eine daraus abgeleitete Anforderungsanalyse zur strukturierten Beschreibung der Systemfunktionen
- Entwürfe für die Benutzeroberfläche (UI) zur Visualisierung der Interaktion mit dem System
- die eingesetzten Hardwarekomponenten zur Umsetzung von Steuerung und Sensorik
- sowie Vereinbarungen zur Zusammenarbeit und Umsetzung innerhalb der Projektgruppe

Ziel dieser Dokumentation ist es, eine transparente und nachvollziehbare Entwicklung, das Testen sowie die Qualitätssicherung des Systems zu schaffen.

Vorgehensweise

Im Rahmen des Projekts wurde ein agiles Vorgehen gewählt, um flexibel auf neue Erkenntnisse, technische Herausforderungen und Änderungen in den Anforderungen reagieren zu können.

Dabei wurde bei der Zusammenarbeit auf regelmäßige Abstimmungen geachtet. In wöchentlichen Meetings wurde der aktuelle Stand des Projekts gemeinsam reflektiert. Dabei lag der Fokus insbesondere auf dem Fortschritt der einzelnen Aufgaben, bestehenden Herausforderungen sowie möglichem Unterstützungsbedarf innerhalb der Gruppe. Auf Basis dieser Abstimmungen wurden die nächsten Arbeitsschritte geplant. Falls erforderlich, erfolgte eine Anpassung der Prioritäten, insbesondere dann, wenn sich Abhängigkeiten zwischen einzelnen Anforderungen oder unerwartete Schwierigkeiten ergeben haben.

Zur Organisation der Aufgaben wurde zunächst die Plattform *Monday* genutzt. Im Laufe des Projekts wurde jedoch zu einem *Kanban-Board* gewechselt, das direkt in GitHub integriert ist. Dieser Wechsel erfolgte, da sich bei der bisherigen Lösung Einschränkungen gezeigt haben und GitHub eine bessere Verknüpfung zwischen Aufgaben und Code ermöglicht. Das Kanban Board diente als zentrale Übersicht über alle Aufgaben. Dadurch war jederzeit transparent nachvollziehbar, welche Aufgaben sich in Bearbeitung befinden, bereits abgeschlossen wurden oder als nächstes anstehen.

Die Entwicklung des Codes erfolgte über GitHub, das als zentrale Plattform für die Versionsverwaltung genutzt wurde. Die Implementierung neuer Funktionalitäten erfolgte in separaten Branches, wodurch paralleles Arbeiten ermöglicht wurde. Erst nach erfolgreicher Entwicklung und Überprüfung wurde der Code in den Main-Branch integriert.

Neben den regelmäßigen Meetings wurde zusätzlich über Messenger-Dienste kommuniziert. Diese ermöglichten eine schnelle Abstimmung bei kurzfristigen Fragen, das Teilen von Zwischenständen und Erfolgen sowie das unmittelbare Reagieren auf Problemen.

In der Mitte des Projekts wurde eine Freeze-Phase durchgeführt. Durch die ausführliche Dokumentation konnte im Anschluss an diese Phase jedoch strukturiert weitergearbeitet werden.

Das gewählte Vorgehen erwies sich insgesamt als effizient und zielführend. Durch die Kombination aus regelmäßiger Abstimmung, strukturierter Aufgabenverwaltung und direkter Kommunikation konnte eine koordinierte Zusammenarbeit sichergestellt werden. Gleichzeitig ermöglicht der agile Ansatz eine flexible Anpassung an neue Herausforderungen während des Projektverlaufs.

User Stories

Die folgenden User Stories bilden die Grundlage für die funktionalen Anforderungen des Projekts R.E.D. – Real-time Environment Driver. Sie beschreiben die Anforderungen aus Sicht der Endnutzer und stellen sicher, dass die Entwicklung konsequent nutzerzentriert erfolgt.

Die User Stories folgen dem in der Vorlesung vermittelten Schema:

„Als [Rolle] möchte ich [Ziel], um [Nutzen].“

Dieses Format ermöglicht eine klare Beschreibung der gewünschten Funktionalitäten, ohne frühzeitig technische Details vorwegzunehmen. Dadurch wird eine strukturierte Überführung in die Systemarchitektur erleichtert.

Zusätzlich wurden die User Stories priorisiert, wobei eine Skala von 1 bis 5 verwendet wird (1 = höchste Priorität, 5 = niedrigste Priorität). Zur besseren Planbarkeit wird außerdem eine Aufwandschätzung für jede User Story angegeben. Im Verlauf des Projektes wurde die Priorisierung der User Stories iterativ angepasst, da sich im Entwicklungsprozess funktionale Abhängigkeiten zwischen den Anforderungen gezeigt haben.

Ergänzend werden User Cards verwendet, die zentrale Interaktionen visuell darstellen und die Kommunikation innerhalb der Gruppe unterstützen. Diese enthalten zudem Akzeptanzkriterien, anhand derer die Erfüllung einer Anforderung überprüft werden kann.

Titel:	Fernsteuerung
Beschreibung:	Als Nutzer möchte ich das Auto aus einer beliebigen Position in einem durchschnittlichen Raum (circa 25 Quadratmeter) mithilfe eines Modus in einer Applikation fernsteuern können, um den Weg des Autos vorzugeben.
Schätzung:	6
Priorität:	2
Titel:	Verbindung der Applikation mit dem Auto
Beschreibung:	Als Nutzer möchte ich auf meinem mobilen Endgerät eine Applikation aufrufen können, um mich mithilfe von WLAN mit dem Auto sowie den anderen Komponenten verbinden.
Schätzung:	5
Priorität:	1
Titel:	Wechseln der Modi im Benutzerinterface
Beschreibung:	Als Nutzer möchte ich in der Applikation in einem Menu zwischen den verschiedenen Modi wechseln können, um die unterschiedlichen Aspekte der Applikation verwenden zu können.
Schätzung:	2
Priorität:	2
Titel:	Rechts- und Linkshänder Modus
Beschreibung:	Als Nutzer möchte ich in der Applikation zwischen einem Rechts- und einem Linkshändermodus wechseln können, um sowohl als Rechts- als auch als Linkshänder die Steuerung bedienen zu können.
Schätzung:	1
Priorität:	2

Titel:	Semi-autonomes Fahren
Beschreibung:	Als Nutzer möchte ich in dem Benutzerinterface der Applikation einen Modus einstellen können, in welchem das Auto mithilfe eines Sensors die Umgebung erkennt und vor Kollision mit einem Gegenstand oder Lebewesen (wie Katzen) stoppt oder ggf. ausweicht, um keine Beschädigungen an Gegenständen im Raum zu erzeugen.
Schätzung:	8
Priorität:	3
Titel:	Karte des Raums
Beschreibung:	Als Nutzer möchte ich in der Applikation einen Modus aufrufen, in welchem das Auto den Raum scannt und in dem die Karte im Anschluss in dem Benutzerinterface angezeigt wird, um anschließend auf dieser Karte einen Weg einzeichnen zu können, dem das Auto folgt.
Schätzung:	4
Priorität:	4
Titel:	autonomes Fahren
Beschreibung:	Als Nutzer möchte ich im Benutzerinterface der Applikation einen Modus einstellen, in welchem das Auto mithilfe eines Sensors die Umgebung erkennt und selbstständig in einer zuvor definierten Umgebung fährt, um das Auto nicht selbstständig steuern zu müssen.
Schätzung:	5
Priorität:	4
Titel:	Erklärung der Funktionsweisen
Beschreibung:	Als Nutzer möchte ich auf ein Button gehen, wo mir die Funktionsweise der Steuerung des Autos erklärt wird, um anschließend selbst in der Lage zu sein, das Auto entsprechend zu steuern.
Schätzung:	3
Priorität:	5

User Cards

Titel: Fernsteuerung		ID: F24
User Story Als Nutzer möchte ich das Auto aus einer beliebigen Position in einem durchschnittlichen Raum (circa 25 Quadratmeter) mithilfe einer Applikation fernsteuern können, um den Weg des Autos vorzugeben.	Priorität 2	Type ☒ Kunde ☒ Produkt ☐ System ☐ Prozess ☐ Statistik
Akzeptanz Kriterium Das Auto fährt in die Richtung, welche von mir im Benutzerinterface vorgegeben wird und beschleunigt bzw. bremst innerhalb von 500ms, wenn ich auf die entsprechenden Buttons in der Applikation gehe.	Schätzung 4	

Titel: Verbindung der Applikation mit dem Auto		ID:VAA15
User Story Als Nutzer möchte ich auf meinem mobilen Endgerät eine Applikation aufrufen können, um mich mithilfe von WLAN mit dem Auto sowie den anderen Komponenten verbinden.	Priorität 1	Type ☒ Kunde ☒ Produkt ☐ System ☐ Prozess ☐ Statistik
Akzeptanz Kriterium Ich kann die Applikation aufrufen und kann dort auf das Auto zugreifen und dem Auto Befehle geben.	Schätzung 5	

Titel: Wechseln der Modi im Benutzerinterface		ID: WMB22
User Story Als Nutzer möchte ich in der Applikation in einem Menu zwischen den verschiedenen Modi wechseln können, um die unterschiedlichen Aspekte der Applikation verwenden zu können.	Priorität 2	Type ☒ Kunde ☒ Produkt ☐ System ☐ Prozess ☐ Statistik
Akzeptanz Kriterium Ich kann in der Applikation in ein Menu gehen und zwischen den verschiedenen Modi wechseln. Wenn ich auf einen bestimmten Modus klicke, gelange ich auf eine neue Unterseite, in welcher der entsprechende Modus dargestellt ist.	Schätzung 2	

Titel: Rechts- und Linkshänder Modus ID: RLM21		
User Story Als Nutzer möchte ich in der Applikation zwischen einem Rechts- und einem Linkshändermodus wechseln können, um sowohl als Rechts- als auch als Linkshänder die Steuerung bedienen zu können. Akzeptanz Kriterium Ich kann in der Applikation auf einen Button klicken, wodurch die Steuerungselemente getauscht werden in meinem Fahrmodus.	Priorität 2 Schätzung 1	Type ☒ Kunde ☒ Produkt ☐ System ☐ Prozess ☐ Statistik

Titel: Semi-autonomes Fahren ID: SAF38		
User Story Als Nutzer möchte ich in dem Benutzerinterface der Applikation einene Modus einstellen können, in welchem das Auto mithilfe eines Sensors die Umgebung erkennt und vor Kollision mit einem Gegenstand oder Lebewesen stoppt oder ggf. ausweicht, um keine Beschädigungen an Gegenständen im Raum zu erzeugen. Akzeptanz Kriterium Ich kann den Modus Fahren aufrufen, in welchem das Auto automatisch stoppt oder ausweicht, bevor ich gegen einen Gegenstand fahre.	Priorität 3 Schätzung 8	Type ☒ Kunde ☒ Produkt ☐ System ☐ Prozess ☐ Statistik

Titel: Karte des Raums ID: KR54		
User Story Als Nutzer möchte ich in der Applikation einen Modus aufrufen, in welchem das Auto den Raum scannt und in dem die Karte im Anschluss in dem Benutzerinterface angezeigt wird, um anschließend auf dieser Karte einen Weg einzeichnen zu können, dem das Auto folgt. Akzeptanz Kriterium Ich kann einen Kartenmodus aufrufen, in welchem eine 2D Karte des Raumes angezeigt wird. Auf dieser Karte kann ich einen Weg einzeichnen und das Auto fährt dem Weg im Raum nach.	Priorität 4 Schätzung 6	Type ☒ Kunde ☒ Produkt ☐ System ☐ Prozess ☐ Statistik

Titel: autonomes Fahren		ID: AF45
User Story Als Nutzer möchte ich im Benutzerinterface der Applikation einen Modus einstellen, in welchem das Auto mithilfe eines Sensors die Umgebung erkennt und selbstständig in einer zuvor definierten Umgebung fährt, um das Auto nicht selbstständig steuern zu müssen. Akzeptanz Kriterium Ich kann einen Modus aufrufen, in welchem das Auto in dem Raum selbstständig ohne Interaktion von mir fährt.	Priorität 4 Schätzung 5	Type ☒ Kunde ☒ Produkt ☐ System ☐ Prozess ☐ Statistik

Titel: Erklärung der Funktionsweisen		ID: EF53
User Story Als Nutzer möchte ich auf ein Button gehen, wo mir die Funktionsweise der Steuerung des Autos erklärt wird, um anschließend selbst in der Lage zu sein, das Auto entsprechend zu steuern. Akzeptanz Kriterium Ich kann auf einen Button klicken und es erscheint eine Erklärung, wie die Steuerung des Autos in der Applikation funktioniert (Gas, Bremse, Steuerung).	Priorität 5 Schätzung 3	Type ☒ Kunde ☒ Produkt ☐ System ☐ Prozess ☐ Statistik

Anforderungsanalyse, abgeleitet aus den User Stories

Die Anforderungsanalyse bildet die Grundlage für die technische Umsetzung des Projektes R.E.D. Sie überführt die aus den User Stories abgeleiteten Nutzerbedürfnisse in präzise und überprüfbare Systemanforderungen.

Die Anforderungen sind in der nachstehenden Tabelle zusammengefasst und priorisiert (1 = höchste, 5 = niedrigste Priorität). Jede Anforderung ist eindeutig formuliert und kann im weiteren Projektverlauf überprüft und getestet werden.

Zur Sicherstellung der Nachvollziehbarkeit entsprechen die IDs der Anforderungen den IDs der zugehörigen User Cards. Dadurch wird eine direkte Rückverfolgbarkeit zwischen Nutzeranforderung und Systemfunktion gewährleistet.

ID	Titel	Anforderung	Priorität
VAA15	Verbindung der Applikation mit dem Auto	Das System muss eine Verbindung zwischen der Applikation und dem Auto herstellen. Die Verbindung soll über einen vom Fahrzeug bereitgestellten Access Point erfolgen, bei dem sich das Endgerät anmelden kann.	1
F24	Fernsteuerung	Das System muss die Steuerung des Fahrzeugs über das Benutzerinterface ermöglichen. Dazu müssen Richtungssteuerung (links/rechts), Beschleunigung (Gas) und Bremsen verfügbar sein. Rückwärtsfahren soll durch ein längeres Betätigen der Bremse ausgelöst werden können.	2
WMB22	Wechseln der Modi im Benutzerinterface	Das System muss ein Auswahlmenü bereitstellen, über das der Nutzer zwischen	2

		den Modi Fahren, Strecke zeichnen und autonomes Fahren wechseln kann.	
RLM21	Rechts- und Linkshänder Modus	Das System soll einen Umschaltmechanismus bereitstellen, der die Position der Lenkungs- und Gas/Bremse-Bedienelemente vertauscht.	2
SAF38	Semi-autonomes Fahren	Das System soll Sensordaten erfassen und bei aktivem semi-autonomen Modus auf erkannte Hindernisse reagieren (Stoppen oder Ausweichen), während eine manuelle Steuerung des Autos weiterhin möglich ist.	3
AF45	Autonomes Fahren	Das System soll einen autonomen Fahrmodus mit Start/Stopp bereitstellen. Sensordaten müssen erfasst werden, wodurch die selbstständige Navigation in einem definierten Bereich ermöglicht werden soll.	4
KR54	Karte des Raums	Das System soll eine Kartenansicht des gescannten Raums anzeigen und muss das Zeichnen einer Route per Touch erlauben, der das Fahrzeug anschließend folgt.	4
EF53	Erklärung der Funktionsweisen	Das System soll kontextbezogene Hilfen (z.B. Pop-ups) zur Erklärung der Bedienung und Funktionen bereitstellen.	5

Vereinbarungen zur Umsetzung

Zur Umsetzung der Anforderungen wurden folgende technische und funktionale Vereinbarungen getroffen:

Verbindung der Website oder Applikation mit dem Auto

Die Verbindung erfolgt über einen vom Fahrzeug bereitgestellten WiFi Access Point. Das Endgerät der Nutzer verbindet sich direkt mit diesem Netzwerk.

Fernsteuerung

Die manuelle Steuerung erfolgt über die mobile Applikation. Dabei werden die Steuerbefehle für die Lenkung (links/rechts), Beschleunigung und Bremsen übermittelt.

Hardware

Für die Umsetzung werden folgende Komponenten verwendet:

- ESP-12E Shield Board (elektrische Komponenten)
- ESP8266 (Logik und Access Point)
- LED für Beleuchtung
- Auto
- Optionale Erweiterung: Infrarotsensoren zur Abstandsmessung

Funktionen für die Überprüfung

Für das Überprüfen der Funktionalitäten und Testen der einzelnen Funktionen sollen folgende Funktionen im Entwicklungsprozess zugänglich sein:

- Log-Ansicht
- Deaktivierung der Hinderniserkennung
- Anzeige der gescannten Umgebung

Raum Scannen

Beim Wechsel in den Zeichenmodus soll der Scan automatisch gestartet werden (kein manuelles Starten).

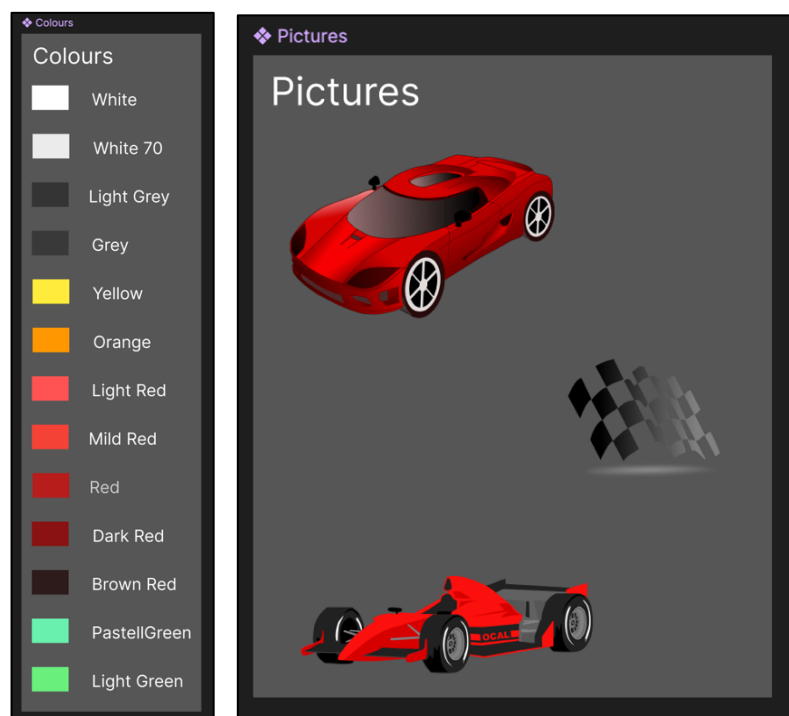
Entwurf

In diesem Kapitel werden die auf Basis der Anforderungen entwickelten Entwürfe beschrieben. Dabei wird darauf geachtet, in dem Entwurf die verschiedenen Anforderungen zu adressieren und konsistent mit den User Stories umzusetzen. Der Fokus liegt dabei auf der konzeptionellen Gestaltung des Benutzerinterfaces, während die Umsetzung und technische Details zur Implementierung im Kapitel Architektur betrachtet werden.

Entwurf des Benutzerinterfaces

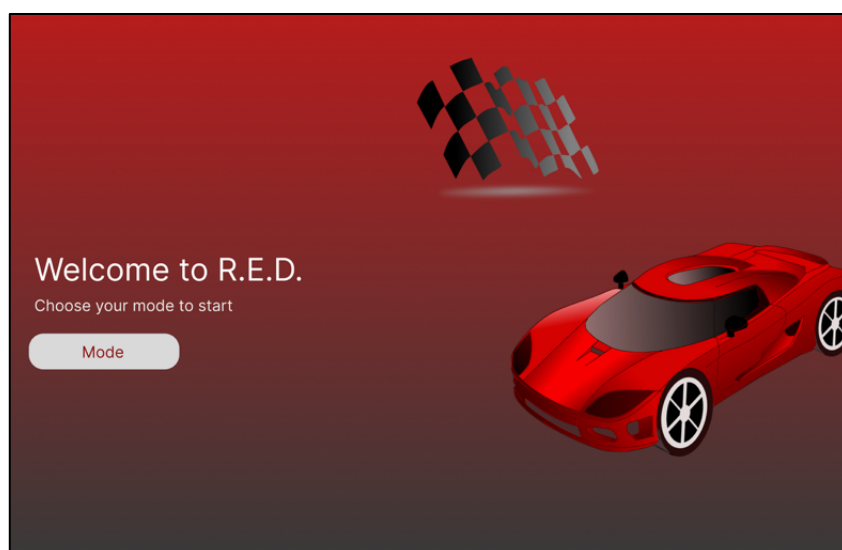
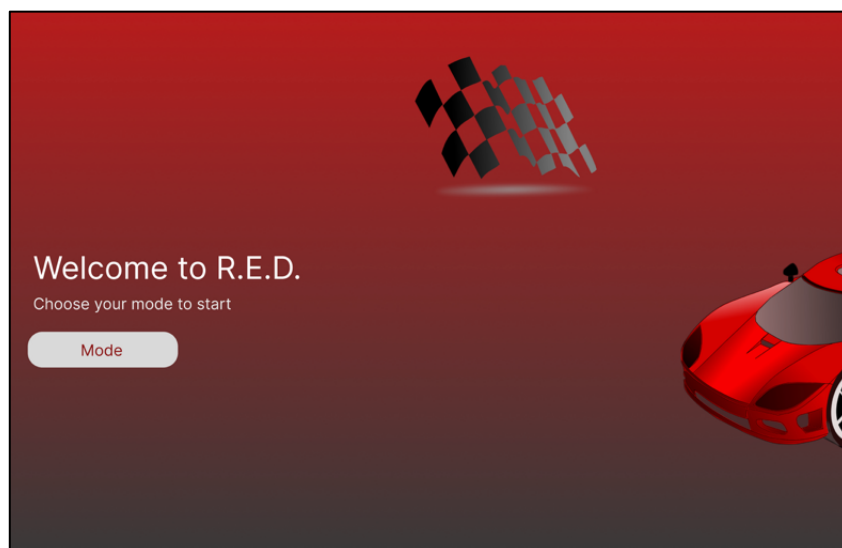
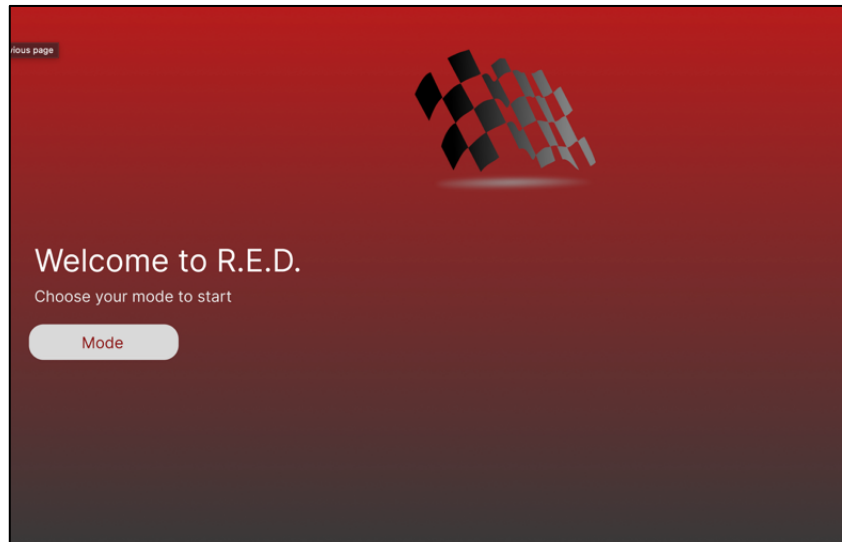
Die Benutzeroberfläche wurde mithilfe des Design-Tools *Figma* entworfen. Dabei lag der Fokus auf einer intuitiven, übersichtlichen und benutzerfreundlichen Interaktion. Im weiteren Projektverlauf wurden die initialen Entwürfe angepasst. Diese Änderungen erfolgten insbesondere zur Verbesserung der Usability sowie zur besseren Ausrichtung auf die Zielgruppe. Dabei zeigte sich, dass eine klare Struktur sowie visuell hervorgehobene Bedienelemente die Verständlichkeit und Bedienbarkeit erhöhen.

Die Farbgestaltung basiert hauptsächlich auf Rottönen in Anlehnung an den Projektnamen R.E.D. Ergänzend wurden Grüntöne als Komplementärfarben eingesetzt, um visuelle Kontraste zu schaffen. Diese sind im Folgenden gezeigt.



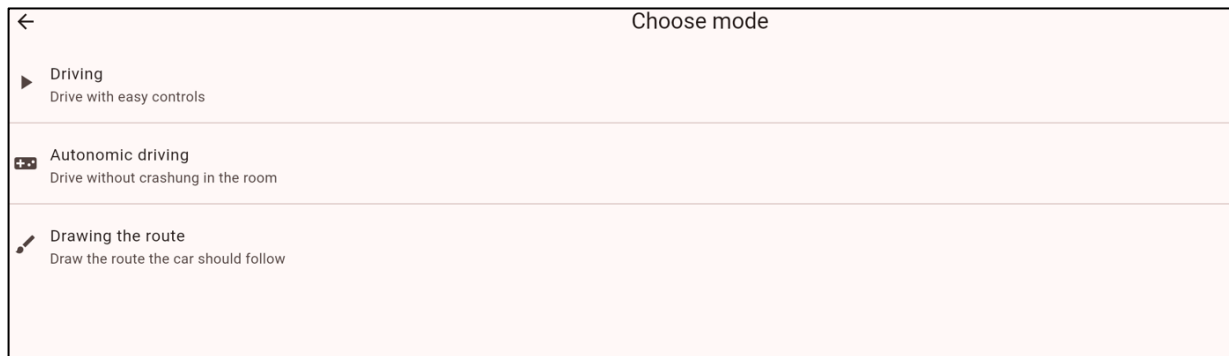
Willkommensseite

Die Willkommensseite dient als Einstiegspunkt der Applikation und soll durch den Einsatz von Animationen einen ansprechenden Einstieg ermöglichen.



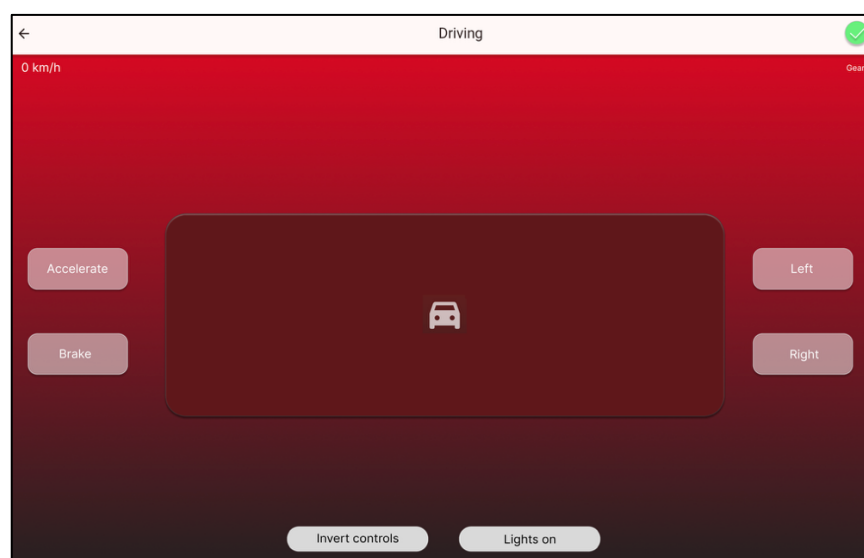
Modusauswahl

Die Modusauswahl stellt eine zentrale Navigationskomponente dar. Auf dieser Seite sollen die Nutzer zwischen den verschiedenen Betriebsmodi wählen können. Diese Auswahl eines Modus führt zur entsprechenden Funktionsseite. Damit soll insbesondere die Anforderung WMB22 umgesetzt werden.

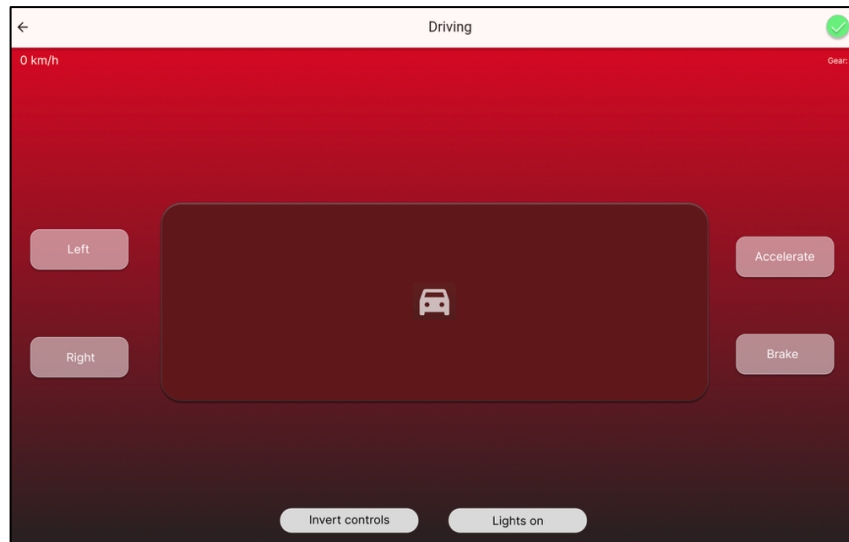


Manueller Fahrmodus

Nach Auswahl des Fahrmodus soll der Nutzer zu einer Steuerungsseite gelangen, über die das Fahrzeug manuell bedient werden kann. Das Benutzerinterface stellt hierfür grundlegende Steuerelemente zur Verfügung. Diese beinhalten die Lenkung (links und rechts), die Beschleunigung und das Bremsen. Zur Berücksichtigung unterschiedlicher Nutzerpräferenzen sind zwei alternative Anordnungen der Bedienelemente vorgesehen. Dadurch wird sowohl Rechts- als auch Linkshändern eine komfortable Nutzung ermöglicht.

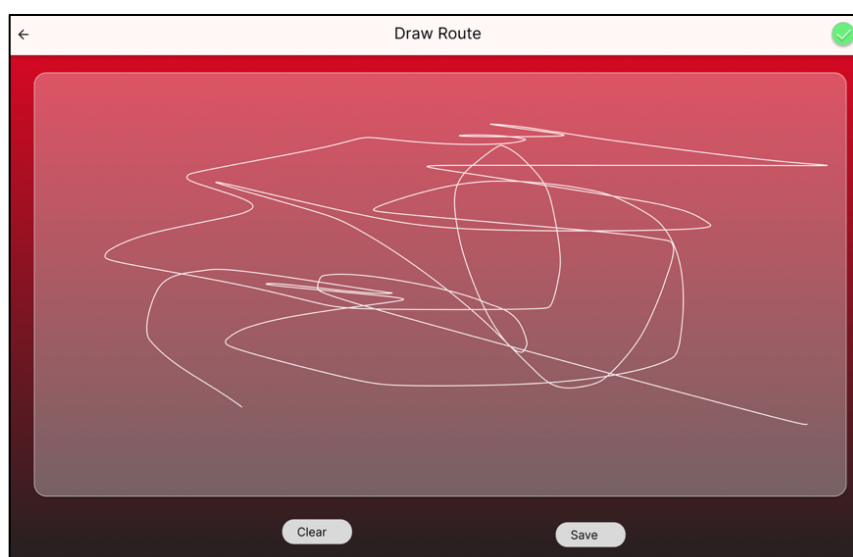


Zusätzlich sind weitere Funktionen vorgesehen, welche die Umschaltung zwischen dem Rechts- und Linkshändermodus und die Anzeige und Steuerung der Fahrzeugbeleuchtung beinhalten. Dieser Modus adressiert insbesondere die Anforderung F24.



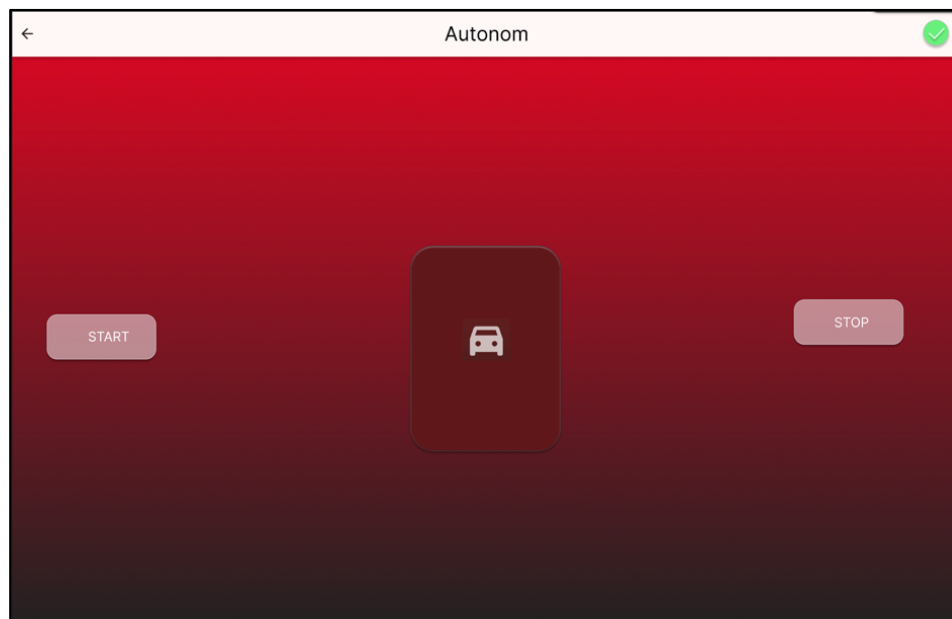
Routenzeichenmodus

Im Routenzeichenmodus soll der Nutzer eine Fahrstrecke auf dem Bildschirm definieren können. Diese Route soll anschließend als Grundlage für die Bewegung des Fahrzeugs dienen. Die Interaktion erfolgt über eine visuelle Zeichenfläche. Zusätzlich ist vorgesehen, die Route zurückzusetzen und neu zu zeichnen. Außerdem soll in diesem Modus der Raum nach Möglichkeit gescannt und die Umgebung anschließend in der Zeichenfläche angezeigt werden. Dieser Modus dient zur Umsetzung der Anforderung KR54.



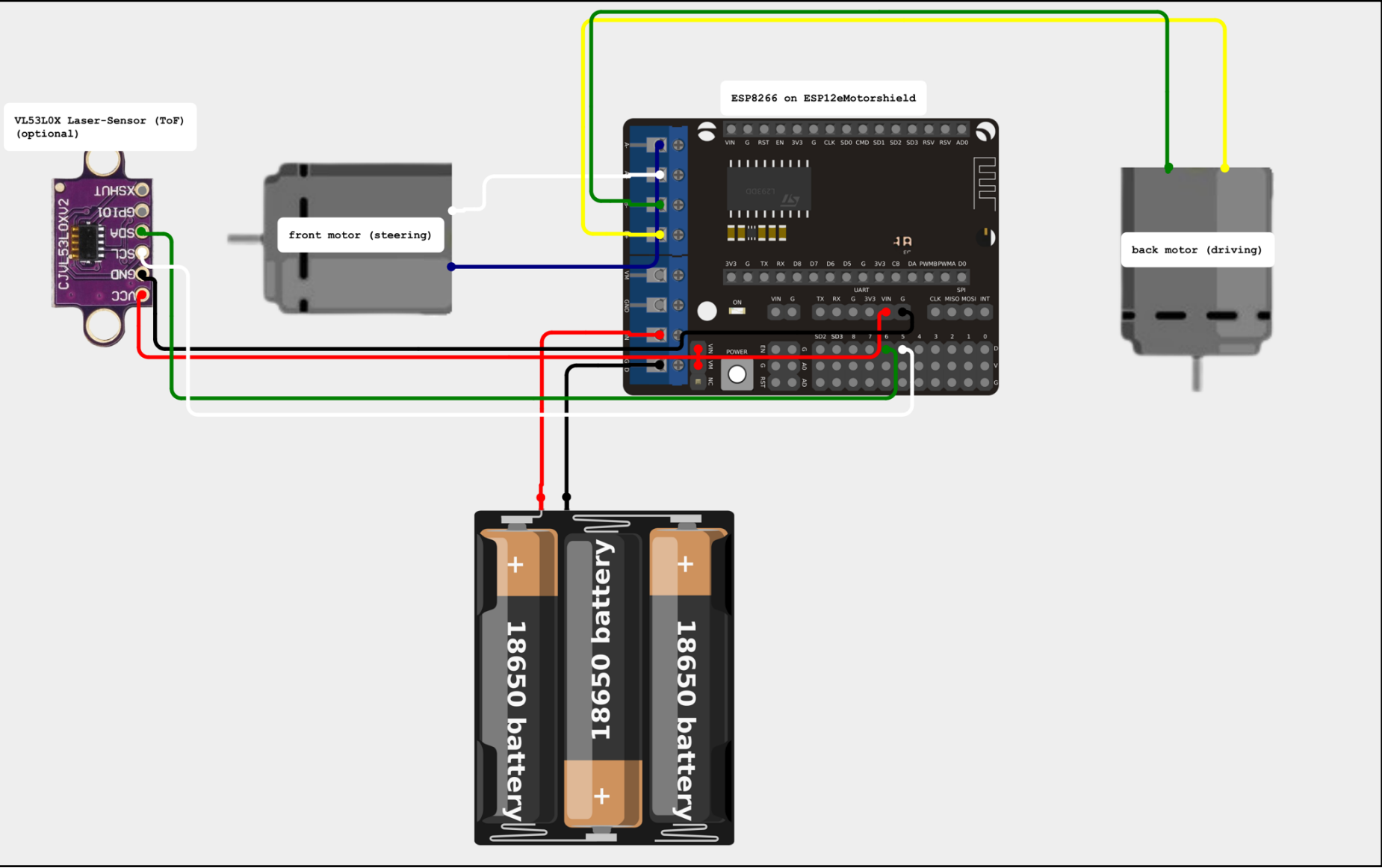
Autonomer Modus

Im autonomen Modus soll das Fahrzeug ohne direkte Interaktion des Nutzers agieren. Grundlage hierfür ist die kontinuierliche Auswertung von Sensordaten, sodass Hindernisse erkannt und entsprechende Reaktionen (zum Beispiel Stoppen oder Ausweichen) ausgelöst werden können. Damit soll die Anforderung AF45 umgesetzt werden.



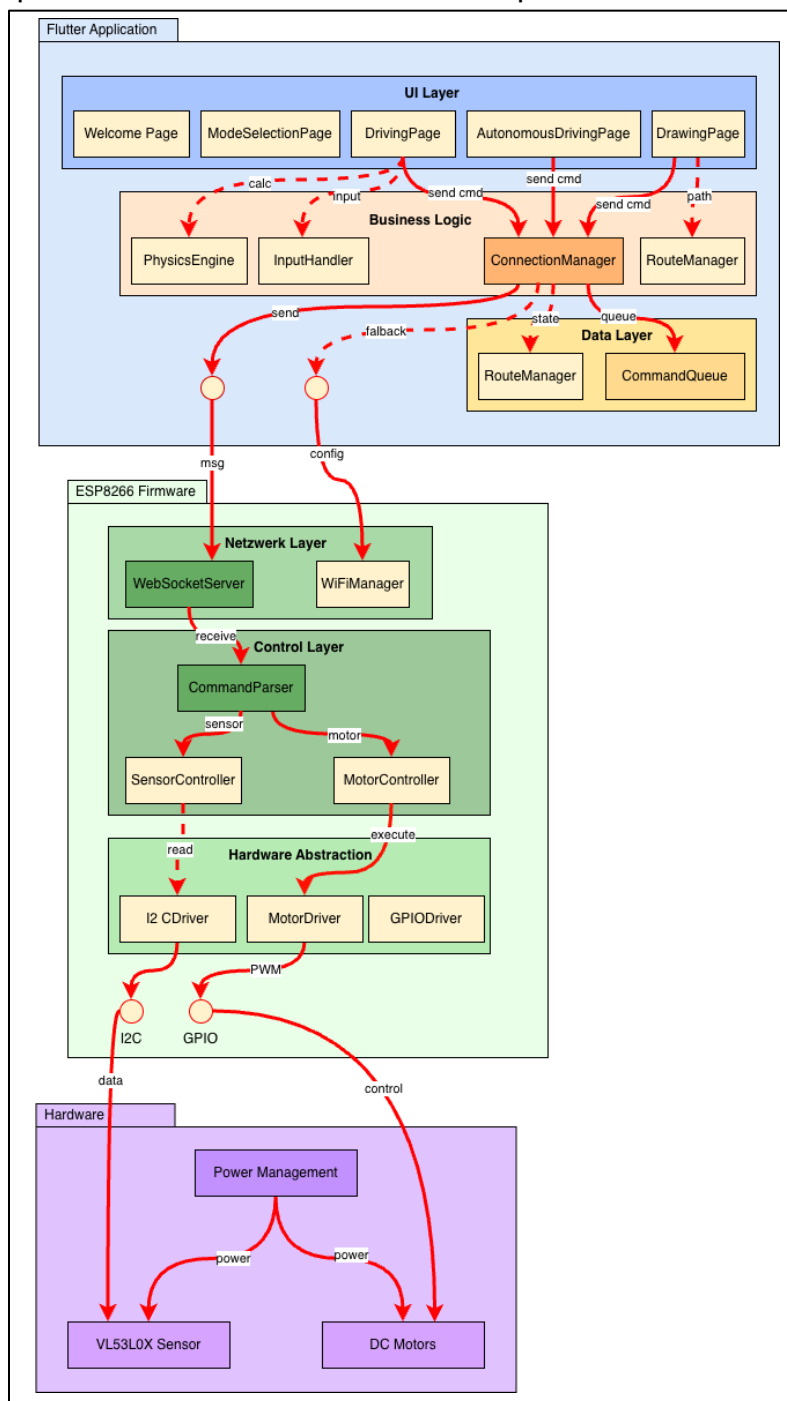
Entwurf der Hardwarekomponenten am Fahrzeug

Für die Umsetzung des Systems wurde ebenfalls ein Entwurf der Hardwarekomponenten erstellt. Dieser beschreibt die Integration der einzelnen Bauteile, insbesondere des Mikrocontrollers, der Sensorik sowie der Motorsteuerung. Die genaue technische Umsetzung wird im Architekturkapitel detailliert beschrieben.



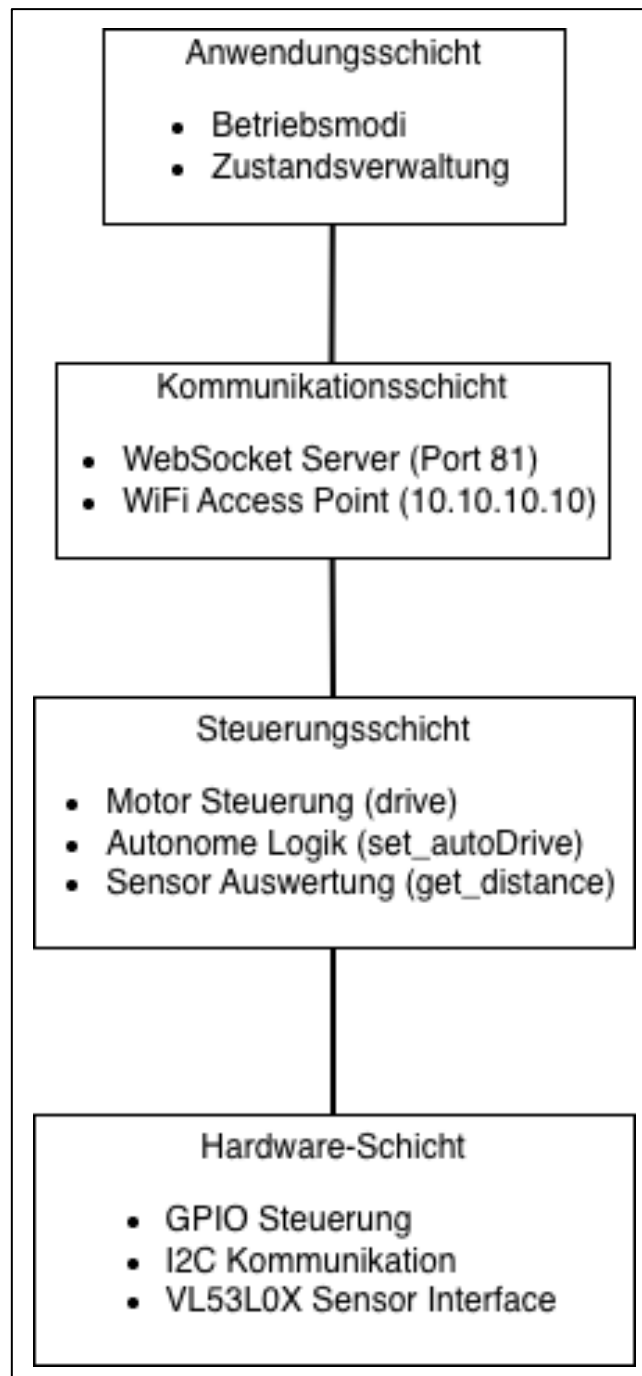
Architektur

Die Architektur des Systems basiert auf einem modularen Aufbau. Dieser lässt sich in drei zentrale Bereiche unterteilen, die mobile Applikation zur Benutzerinteraktion, die Firmware auf dem Mikrocontroller zur Verarbeitung der Steuerbefehle sowie die Hardware zur Ausführung der Bewegung und zur Erfassung von Sensordaten. Das folgende Diagramm stellt die Architektur des Systems dar und zeigt insbesondere die einzelnen Komponenten sowie deren Zusammenspiel.



Softwareschichten

Neben der Gesamtarchitektur wird innerhalb des Systems ein Schichtmodell verwendet, um die Aufgabenbereiche klar voneinander zu trennen. Das folgende Diagramm visualisiert die verschiedenen Softwareschichten und deren Verantwortlichkeiten.



Architektur des Frontends

Das Frontend wurde mit dem Framework Flutter umgesetzt und stellt die zentrale Schnittstelle zwischen Nutzer und System dar.

Die Anwendung ist modular aufgebaut und gliedert ihre Funktionalitäten in mehrere unabhängigen Seiten, die jeweils eine spezifische Aufgabe übernehmen (zum Beispiel Steuerung, Modusauswahl oder Routenzeichnung). Diese Aufteilung ermöglicht eine klare Trennung der Verantwortlichkeiten und erleichtert sowohl die Erweiterung als auch die Wartung der Anwendung.

Die Verwendung von Flutter ermöglicht eine plattformübergreifende Entwicklung für mobile Endgeräte sowie Webanwendungen auf Basis eines gemeinsamen Codes. Die Anwendungslogik wird dabei in der Programmiersprache Dart implementiert.

Schichtmodell des Frontends

Zur Strukturierung der Anwendung wird ein Schichtenmodell verwendet, das die verschiedenen Aufgabenbereiche voneinander trennt.

Die UI-Schicht (Präsentationsschicht) ist für die Darstellung der Benutzeroberfläche verantwortlich und bildet die direkte Schnittstelle zum Nutzer. Sie umfasst sämtliche visuellen Elemente der Anwendung, wie Buttons, Anzeigen und Animationen und ermöglicht die Interaktion mit dem System. Zur Verbesserung des Benutzererlebnisses werden gezielt Animationen eingesetzt, die beispielsweise Zustandsänderungen oder Übergänge zwischen verschiedenen Seiten visuell unterstützen. Dadurch entsteht eine dynamische und ansprechende Benutzeroberfläche. Darüber hinaus ist diese responsiv gestaltet, sodass sie sich an verschiedene Bildschirmgrößen und Endgeräte angepasst.

Weiterhin stellt die Logikschicht eine zentrale Komponente des Schichtmodells dar. Diese Schicht ist für die Verarbeitung von Benutzereingaben verantwortlich, beispielsweise bei Interaktion über Buttons oder andere Steuerelemente. Die Eingaben werden dabei ausgewertet und in entsprechende Systemaktionen umgesetzt. Darüber hinaus übernimmt die Logikschicht die Verwaltung der Anwendungszustände sowie notwendige Berechnungen innerhalb der Applikation.

Auf Basis dieser Zustände werden die Eingaben in konkrete Steuerbefehle für das Fahrzeug übersetzt. Die Logikschicht fungiert somit als Bindeglied zwischen der Benutzeroberfläche und der Kommunikationsschicht.

Als weitere Schicht wird die Datenschicht betrachtet. Diese Schicht ist für die Verwaltung von anwendungsinternen Daten zuständig. Dazu gehören insbesondere die Verarbeitung und Speicherung von Routeninformationen sowie die Organisation von Steuerbefehlen innerhalb der Applikation. Komponenten wie der RouteManager übernehmen die Verwaltung und Verarbeitung von Routen, während Elemente wie die CommandQueue für die strukturierte Verarbeitung und Reihenfolge von Befehlen verantwortlich sind. Die Datenschicht dient somit als Grundlage für die Logikschicht und stellt die benötigten Daten für die weiter Verarbeitung bereit.

Zuletzt existiert eine Kommunikationsschicht, die für die Verbindung zwischen der Applikation und dem Mikrocontroller verantwortlich ist. Diese Schicht übernimmt den Aufbau und die Verwaltung der Verbindung sowie das Senden von Steuerbefehlen an das Fahrzeug. Darüber hinaus stellt sie den aktuellen Verbindungsstatus bereit und ermöglicht eine stabile und zuverlässige Kommunikation zwischen Software und Hardware.

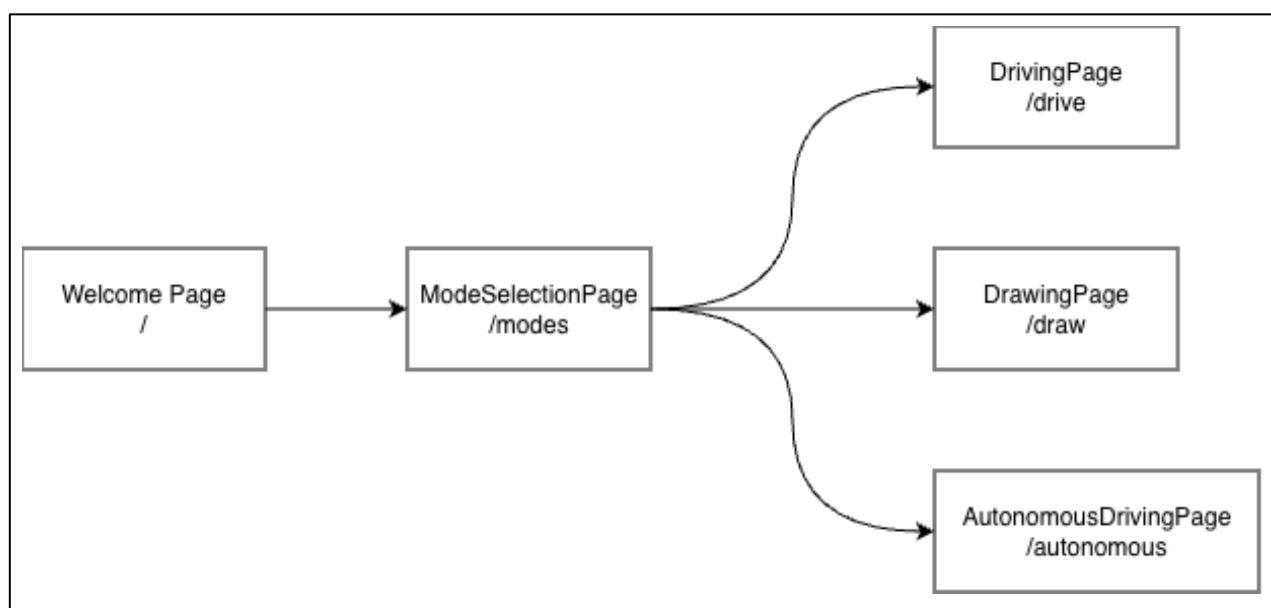
Grundstruktur des Frontends

Das Frontend basiert auf der typischen Architektur einer Flutter-Applikation und bildet die zentrale Steuerungseinheit der Benutzerinteraktion. Die Applikation verfügt über eine zentrale Einstiegskomponente, in der globale Einstellungen wie die Startseite sowie das Routing definiert werden. Über das Routing-System wird die Navigation zwischen den einzelnen Seiten der Anwendung gesteuert. Die Anwendung ist in mehrere Seiten unterteilt, die jeweils eine spezifische Funktionalität abbilden. Diese Struktur ermöglicht eine klare Trennung der Verantwortlichkeiten sowie eine modulare Erweiterung des Systems.

Für das Routing existieren fünf zentrale Pfade, die den jeweiligen Seiten zugeordnet sind:

Route	Klassenname der Seite	Funktionalität der Seite
/	WelcomePage	Willkommensseite, Einstieg in die Applikation
/modes	ModeSelectionPage	Auswahl der verschiedenen Modi
/drive	DrivingPage	Steuerung des Fahrzeugs, Physiksimulation
/autonom	AutonomDrivingPage	Das Auto fährt autonom in dem Raum; es ist keine User Interaktion notwendig
/draw	DrawingPage	Zeichnen einer Route auf dem Bildschirm, dem das Auto folgt

Jede Seite ist als eigenständige Komponente implementiert und verwaltet ihre jeweiligen Zustände unabhängig. Dadurch können Funktionalitäten getrennt entwickelt und einfacher erweitert werden. Zur Umsetzung dynamischer Inhalte werden zustandsbehaftete Komponenten verwendet, die Änderungen im Systemzustand unmittelbar in der Benutzeroberfläche widerspiegeln.



Darstellung der unterschiedlichen Seiten

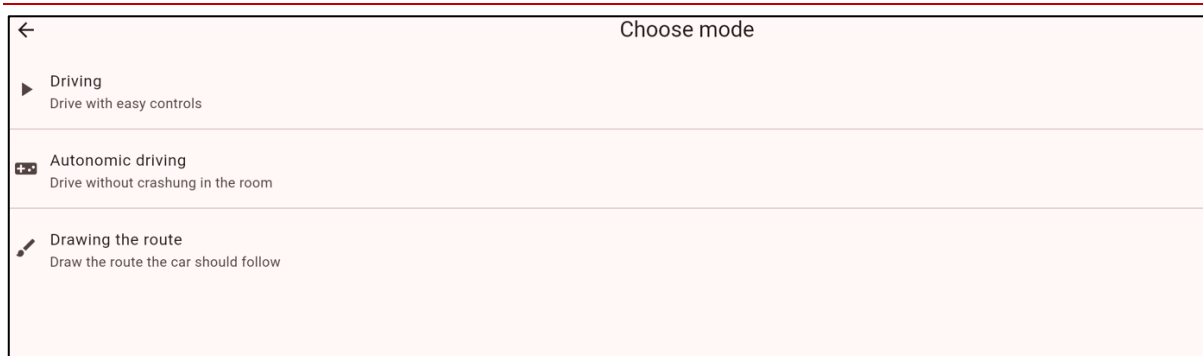
Die Anwendung ist in mehrere zentrale Seiten unterteilt, die jeweils spezifische Funktionalitäten des Systems abbilden. Im Folgenden werden die einzelnen Seiten im Überblick beschrieben, um deren Rolle innerhalb der Gesamtarchitektur sowie deren Beitrag zur Umsetzung der Systemfunktionen zu verdeutlichen.

WelcomePage



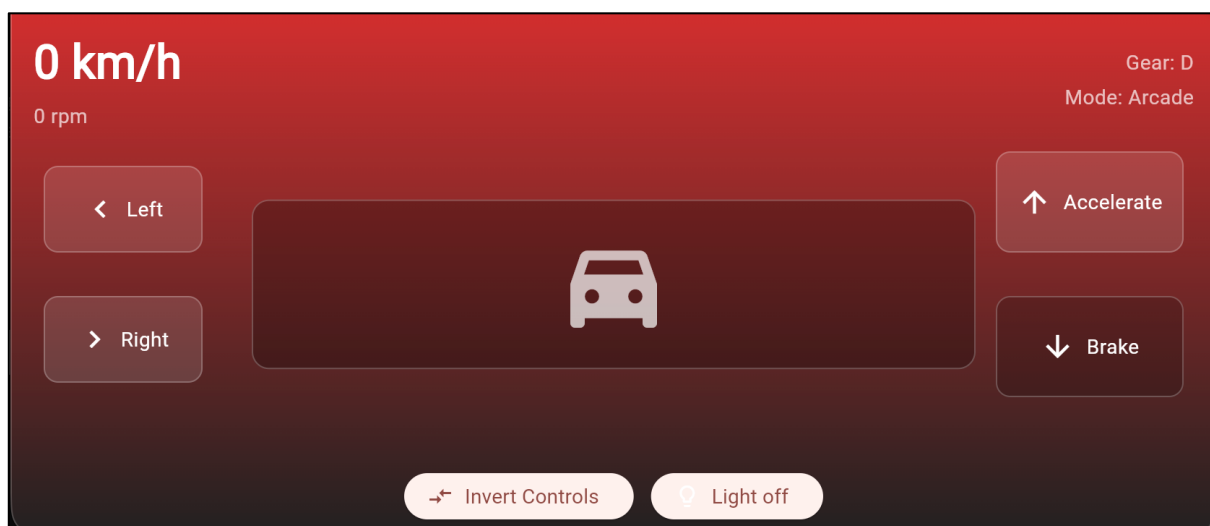
Die WelcomePage stellt den Einstiegspunkt der Anwendung dar und dient als erste Interaktionsfläche für den Nutzer. Sie übernimmt die Aufgabe, den Nutzer in die Anwendung einzuführen und eine Navigation zu den weiteren Funktionen zu ermöglichen. Über ein zentrales Bedienelement gelangt der Nutzer zur Modusauswahl, wodurch die weiteren Funktionalitäten der Applikation erreichbar werden. Dabei werden animierte visuelle Elemente eingesetzt, um den Einstieg in die Anwendung dynamisch darzustellen. Darüber hinaus ist die Benutzeroberfläche responsiv gestaltet, sodass sie sich an unterschiedliche Bildschirmgrößen anpasst.

ModeSelectionPage



Die ModeSelectionPage stellt die zentrale Navigationsseite der Applikation dar. Sie ermöglicht dem Nutzer die Auswahl zwischen den verschiedenen Betriebsmodi des Systems. Zur Verfügung stehen dabei mehrere Modi, die jeweils unterschiedliche Funktionalitäten abbilden. Diese sind ein manueller Fahrmodus zur direkten Steuerung des Fahrzeugs, ein autonomer Modus, in dem das Fahrzeug selbstständig agiert sowie ein Zeichenmodus, in dem eine Route definiert werden kann, der das Fahrzeug anschließend folgt. Die Seite dient somit als zentrale Schnittstelle zur Steuerung der Systemfunktionalität und verbindet die einzelnen Funktionsbereiche der Anwendung miteinander.

DrivingPage



Die DrivingPage ermöglicht die manuelle Steuerung des Fahrzeugs durch den Nutzer und stellt eine zentrale Funktionalität der Applikation dar. Die Steuerung kann sowohl über grafische Bedienelemente als auch über die Tastatur erfolgen. Dabei werden

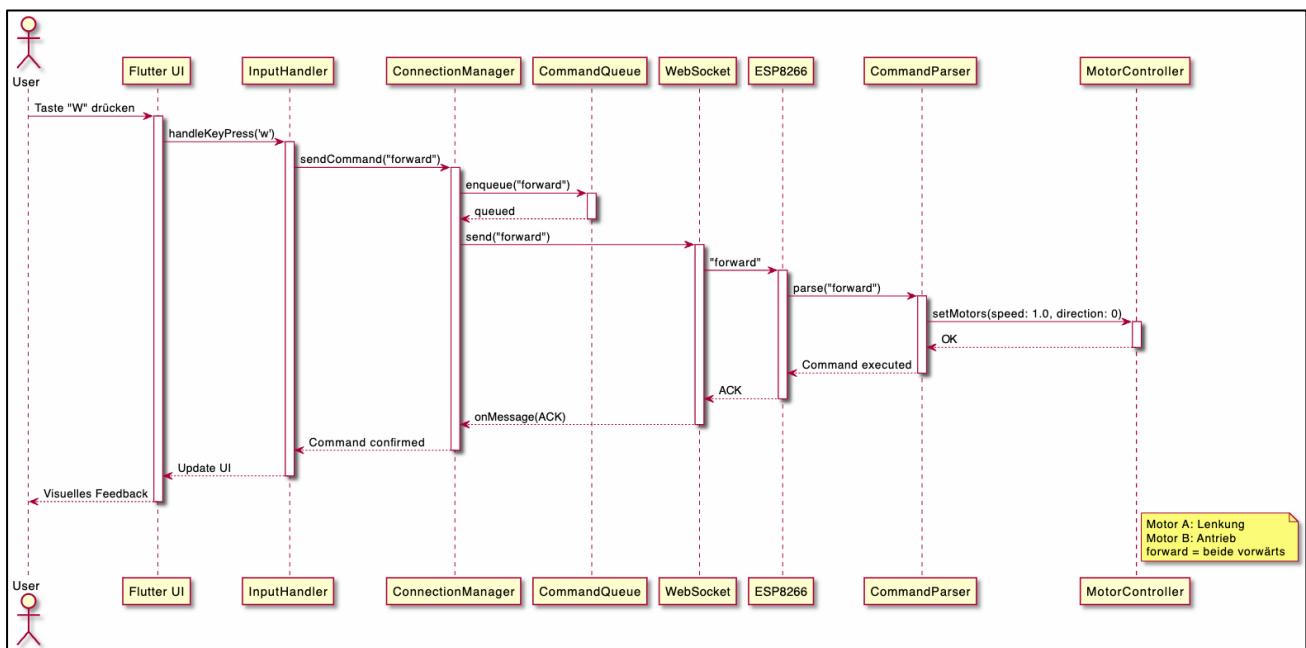
grundlegende Fahrfunktionen wie Beschleunigung, Bremsen sowie Richtungsänderungen unterstützt.

Zusätzlich bietet die Seite erweiterte Funktionen, die das Nutzererlebnis verbessern. Dazu gehören unter anderem die Anpassung der Steuerung für Rechts- und Linkshänder sowie die Steuerung der Fahrzeugbeleuchtung.

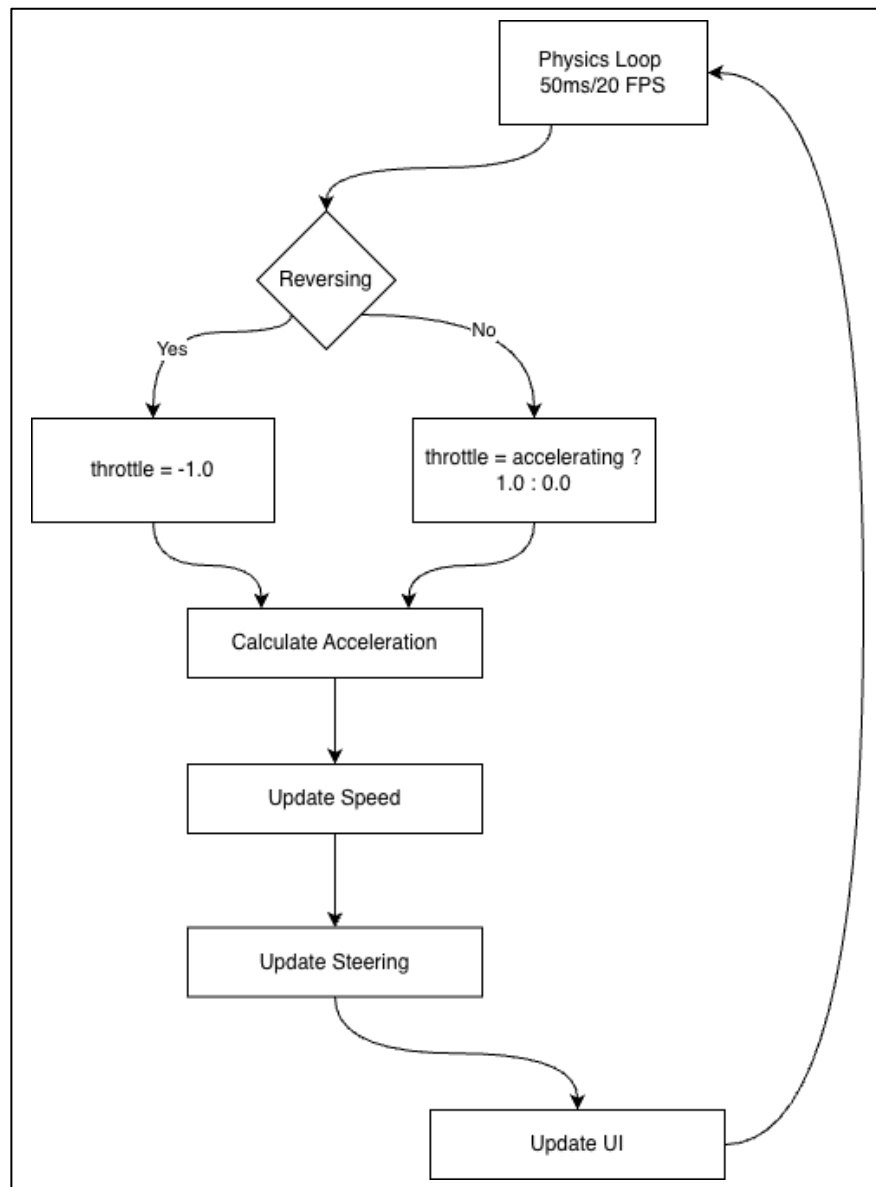
Zur Unterstützung der Benutzerinteraktion wird der aktuelle Zustand des Fahrzeugs visuell dargestellt, beispielsweise durch eine Darstellung der Lenkbewegung. Dadurch erhält der Nutzer unmittelbares Feedback zu seinen Eingaben.

Die DrivingPage vereint somit Steuerung, Zustandsmanagement und visuelles Feedback in einer zentralen Komponente und bildet die Verbindung zwischen Benutzerinteraktion und Fahrzeugsteuerung.

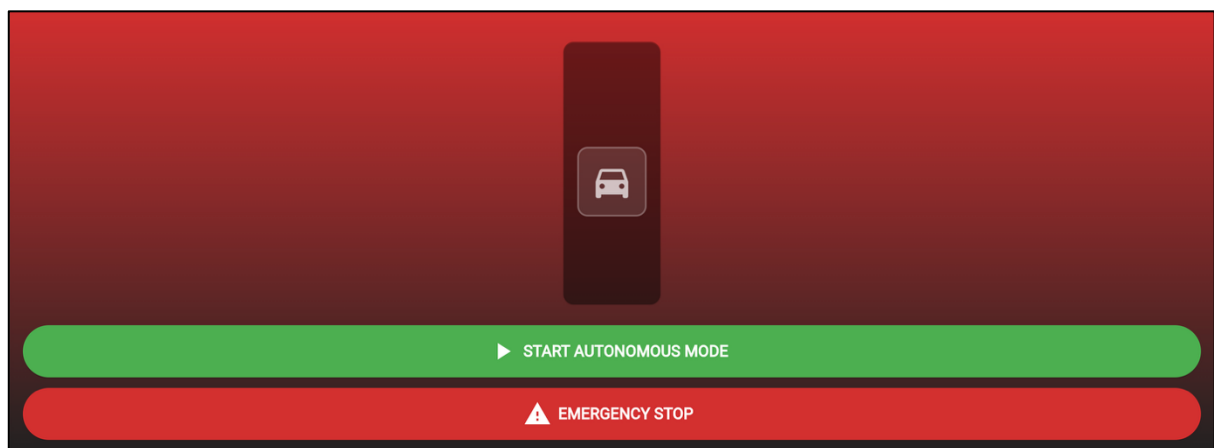
Der genaue Ablauf der Interaktion zwischen Benutzer, Anwendung und Fahrzeug ist im folgenden Sequenzdiagramm dargestellt.



Die interne Verarbeitung der Benutzereingaben sowie die Aktualisierung der Fahrzeugzustände erfolgt in einem zyklischen Ablauf. Das folgende Diagramm veranschaulicht die wichtigsten Schritte dieses Prozesses sowie die Abhängigkeiten zwischen den einzelnen Komponenten.

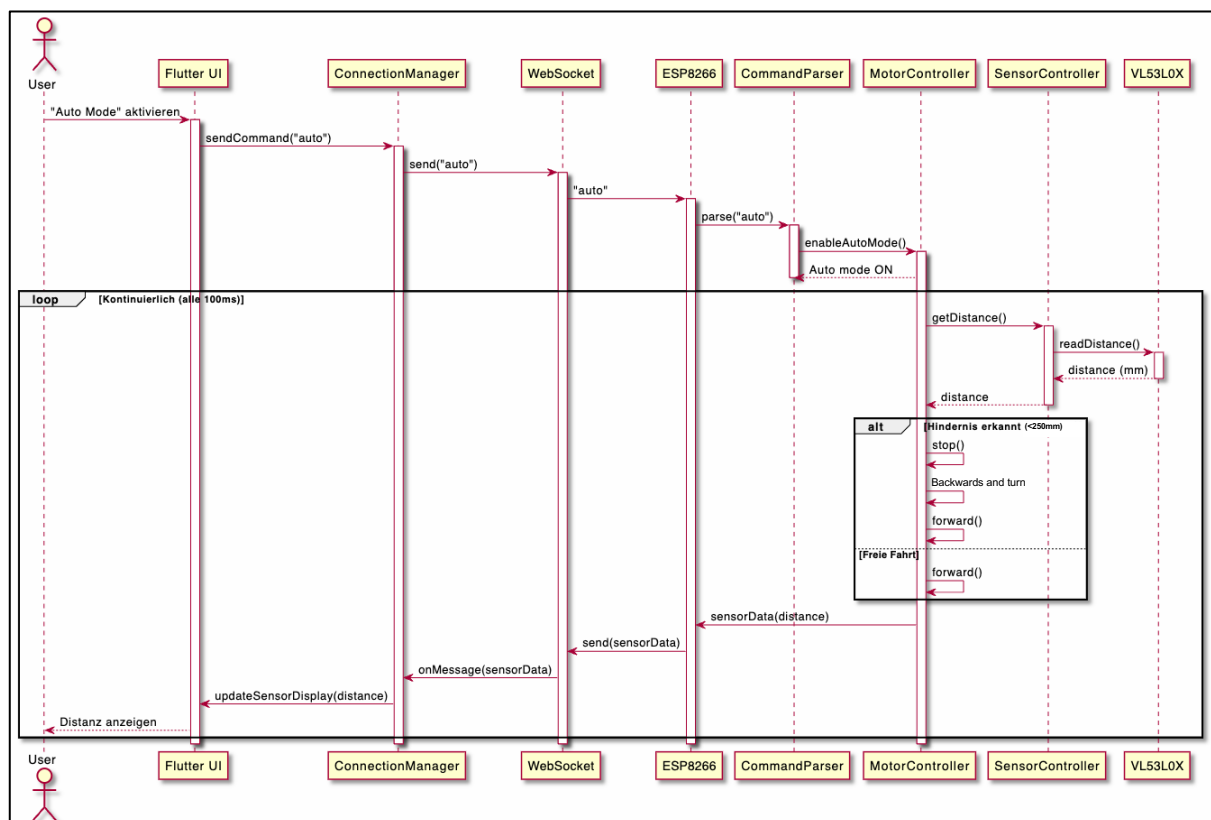


AutonomDrivingPage



Die AutonomDrivingPage ermöglicht den autonomen Betrieb des Fahrzeugs innerhalb einer definierten Umgebung. In diesem Modus fährt das Fahrzeug selbstständig und reagiert auf erkannte Hindernisse, indem es stoppt und die Fahrtrichtung anpasst. Zusätzlich steht ein Emergency-Stop zur Verfügung, mit dem das Fahrzeug jederzeit sofort angehalten werden kann. Ein zentraler Aspekt dieser Funktionalität ist, dass die Logik für das autonome Fahren direkt auf dem Mikrocontroller des Fahrzeugs ausgeführt wird. Dadurch werden Verzögerungen durch die Kommunikation mit der Applikation vermieden und eine schnelle Reaktion auf Umweltveränderungen ermöglicht.

Der Ablauf der Interaktion zwischen Applikation und Fahrzeug ist im folgenden Sequenzdiagramm dargestellt.

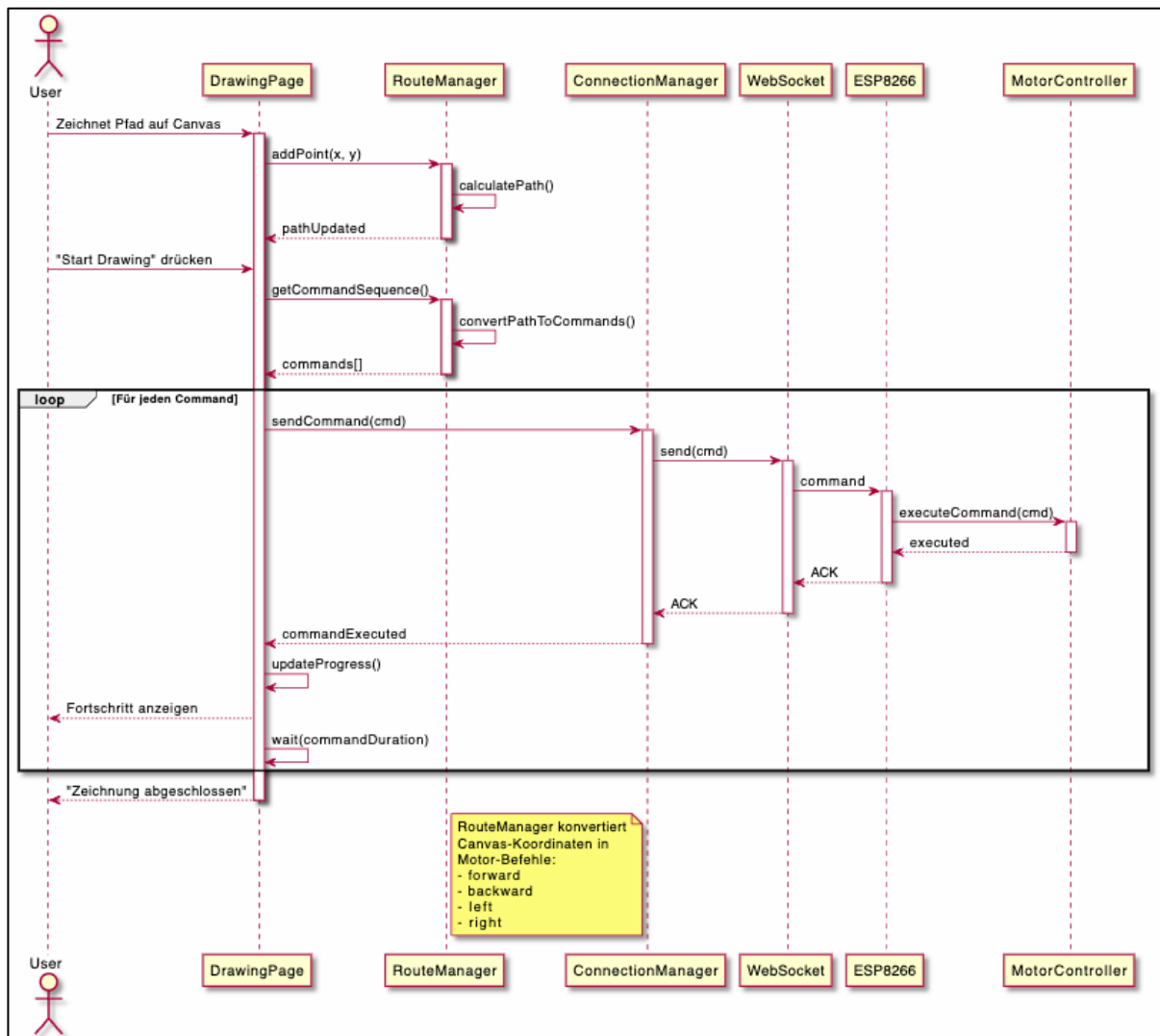


DrawingPage



Die DrawingPage ermöglicht dem Nutzer die Erstellung einer Route, die anschließend vom Fahrzeug abgefahren werden kann. Die Route wird durch direkte Interaktion auf einer Zeichenfläche definiert. Dabei werden die Eingaben des Nutzers erfasst, verarbeitet und in eine strukturierte Form überführt, die als Grundlage für die Fahrzeugsteuerung dient. Auf Basis dieser definierten Route werden entsprechende Steuerbefehle generiert, die dem Fahrzeug eine sequenzielle Bewegung entlang der gezeichneten Strecke ermöglichen. Die Verarbeitung der Route umfasst dabei sowohl die Interpretation der Eingaben als auch die Aufbereitung der Daten für die weitere Nutzung innerhalb der Anwendung.

Der Ablauf der Interaktion und Verarbeitung wird im folgenden Sequenzdiagramm dargestellt.



Kommunikationsintegration

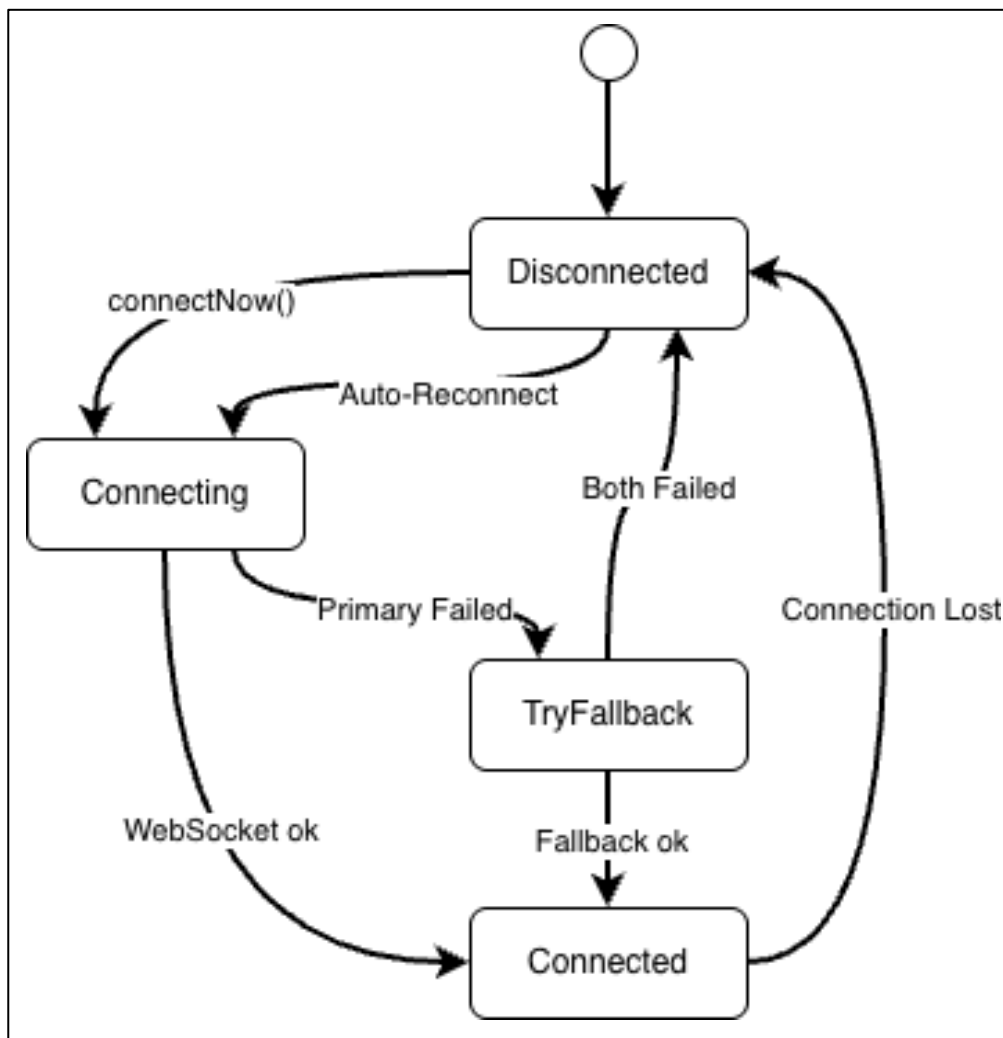
Die Verbindung der Applikation mit dem Auto wird in Flutter mithilfe einer zentralen Komponente, dem ConnectionManager, realisiert. Der ConnectionManager verwaltet die WebSocket-Verbindung zum ESP8266 und stellt die Schnittstelle für die Übertragung von Steuerbefehlen dar. Die Kommunikation erfolgt primär über die WebSocket-Adresse „ws://10.10.10.10:81“, wodurch eine bidirektionale und nahezu latenzfreie Verbindungen ermöglicht wird.

Zur Sicherstellung der Verbindungsrobustheit sind zusätzliche Fallback-Mechanismen implementiert. Dazu gehören eine alternative WebSocket-Verbindung

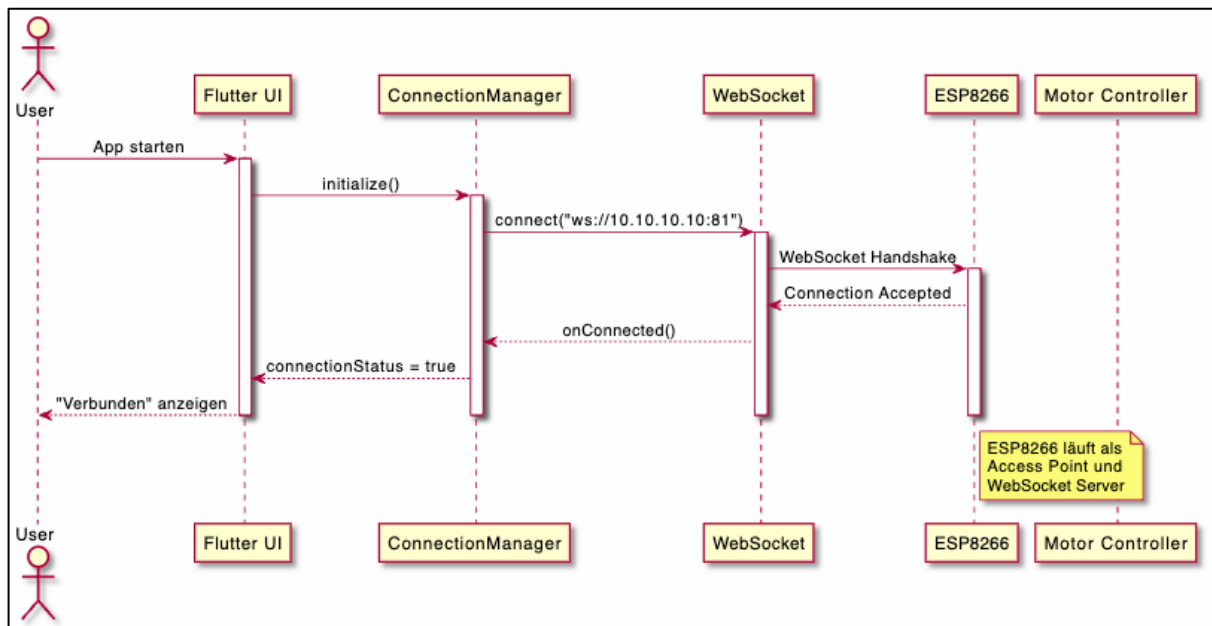
(„ws://localhost:8080/“) sowie eine HTTP-basierte Kommunikation („http://10.10.10.10/move?cmd=“), die im Fehlerfall genutzt werden kann.

Ein weiterer Bestandteil der Kommunikationslogik ist die automatische Wiederherstellung der Verbindung bei Verbindungsabbrüchen, wodurch eine stabile Interaktion zwischen Anwendung und Fahrzeug gewährleistet wird.

Zur Veranschaulichung der Kommunikationslogik werden im Folgenden zwei Diagramme dargestellt. Das erste Diagramm zeigt die möglichen Zustände der Verbindung sowie die Übergänge zwischen diesen, beim Verbindungsaufbau, bei Fehlern oder beim Wechsel auf alternative Kommunikationswege.



Das zweite Diagramm stellt den zeitlichen Ablauf der Kommunikation dar und verdeutlicht die Interaktion zwischen Benutzer, Applikation und Mikrocontroller beim Aufbau der Verbindung und beim Austausch von Steuerbefehlen.



Befehlssystem

Das Befehlssystem bildet die Grundlage für die Steuerung des Fahrzeugs und beschreibt, wie Eingaben aus der Applikation in konkrete Aktionen umgesetzt werden.

Die Kommunikation erfolgt über vordefinierte Steuerbefehle, die von der Applikation an das Fahrzeug übertragen werden. Jeder Befehl entspricht dabei einer bestimmten Aktion des Fahrzeugs.

Die folgenden Befehle werden unterstützt:

Befehl	Bedeutung und Auswirkung
forward	Vorwärts fahren
backward	Rückwärts fahren
left	Links lenken
right	Rechts lenken

stop	anhalten
auto	Autonomer Modus starten
autoStop	Autonomer Modus stoppen
Emergency_stop	Notbremsung

Neben einzelnen Befehlen unterstützt das System auch die Kombination mehrerer Eingaben, um komplexere Bewegungen zu ermöglichen. Dadurch können beispielsweise Vorwärtsbewegung und Richtungsänderung gleichzeitig umgesetzt werden.

Die Zusammenführung und Verarbeitung dieser Befehle erfolgt innerhalb der Anwendung und stellt sicher, dass die Steuerung des Fahrzeugs flexibel und reaktionsfähig umgesetzt werden kann.

Struktur des Backends und der Hardware

In diesem Abschnitt wird die technische Umsetzung des Backends sowie der eingesetzten Hardwarekomponenten beschrieben. Das Backend ist auf dem Mikrocontroller implementiert und übernimmt die Verarbeitung der von der Applikation gesendeten Steuerbefehle sowie die direkte Ansteuerung der Hardware.

Verwendete Technologien: Übersicht

Die folgenden Übersicht zeigt die zentralen Technologien und Komponenten des Systems:

- Microcontroller: ESP8266
- Programmiersprache: C++ (Arduino Framework)
- Kommunikation: Websocket (Port 81)
- Netzwerk: WiFi Access Point
- Sensor: VL53L0X Time-of-Flight Distanzsensor
- Bibliotheken: ESP8266WiFi.h, WebSocketsServer.h, Wire.h, VL53L0X.h

Diese Komponenten bilden die Grundlage für die Kommunikation zwischen Applikation und Fahrzeug sowie für die Verarbeitung von Sensordaten und Steuerbefehlen.

Umsetzung mit C++ und dem Arduino Framework

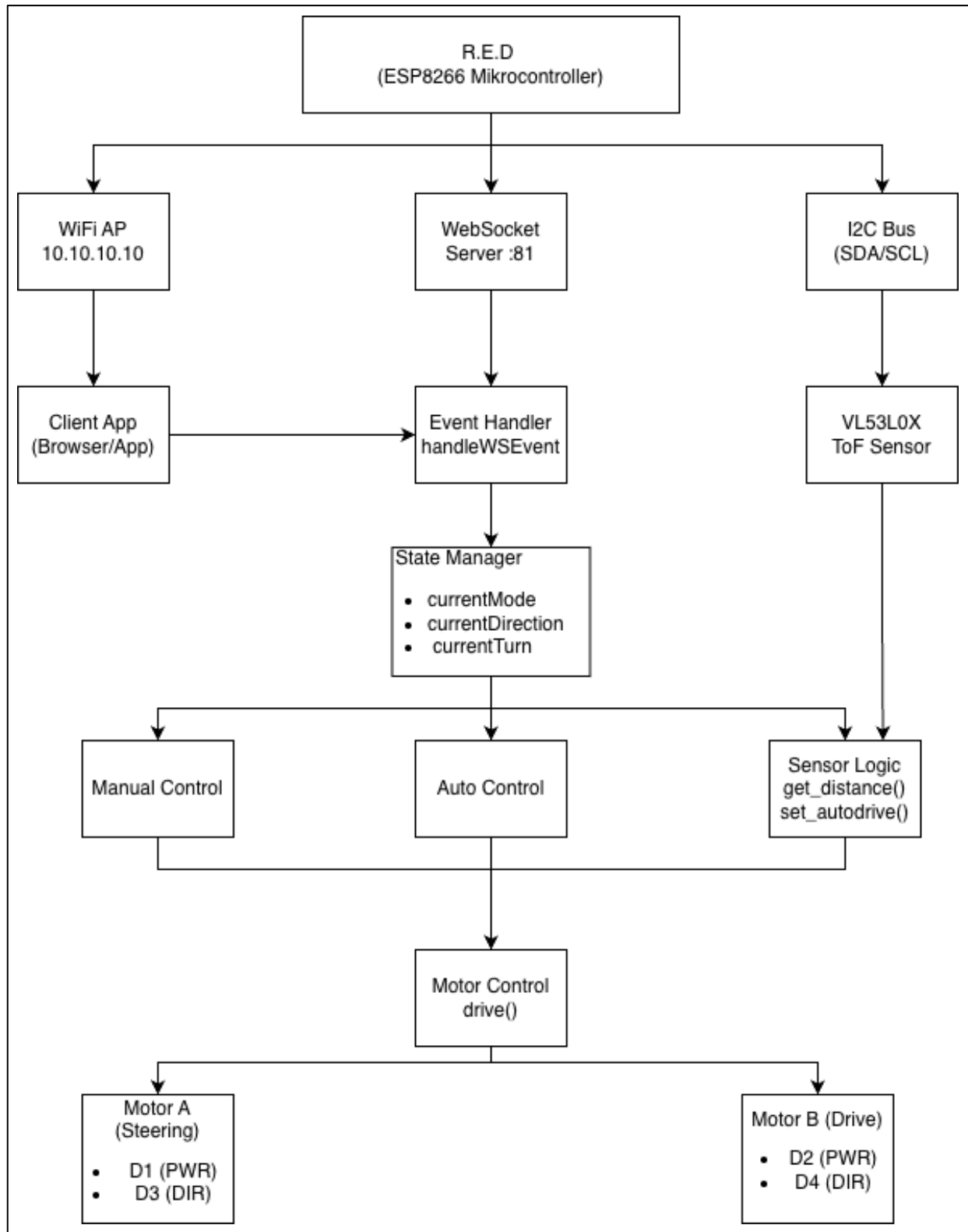
Die Programmierung des Mikrocontrollers erfolgt in C++ unter Verwendung des Arduino Frameworks. Dieses stellt eine abstrahierte Umgebung zur Steuerung von Hardwarekomponenten bereit und ermöglicht die einfache Implementierung von Ein- und Ausgaben, Kommunikation sowie zeitgesteuerten Abläufen.

Die Anwendung auf dem Mikrocontroller folgt dabei dem typischen Aufbau eines eingebetteten Systems mit einer Initialisierungsphase sowie einer kontinuierlich ausgeführten Hauptschleife.

In der Hauptschleife werden eingehende Befehle verarbeitet, Sensordaten ausgelesen und entsprechende Aktionen zur Steuerung des Fahrzeugs ausgelöst.

Systemarchitektur

Die Architektur des Systems ist im folgenden Diagramm und zeigt das Zusammenspiel zwischen Applikation, Mikrocontroller und Hardwarekomponenten.



Hardware Spezifikationen

Dieser Abschnitt beschreibt die im System eingesetzten Hardwarekomponenten sowie deren Funktion innerhalb der Gesamtarchitektur.

Komponenten

Die folgende Übersicht zeigt die zentralen Hardwarekomponenten des Fahrzeugs:

Komponente	Modell	Funktion
Mikrocontroller	ESP8266	Hauptsteuerung
Distanzsensor	VL53L0X	Time-of-Flight Messung
Motor A	DC Motor	Lenkung (links/rechts)
Motor B	DC Motor	Antrieb (vor/zurück)

Pin Belegung

Zur Ansteuerung der Hardwarekomponenten wird der Mikrocontroller über definierte GPIO-Pins konfiguriert. Die Zuordnung der einzelnen Pins ist funktional strukturiert und orientiert sich an den jeweiligen Aufgaben der Komponenten.

Motor A (Lenkung):

Pin	GPIO	Farbe	Funktion	Logik
D1	GPIO5	Weiß	Motor A Ein/Aus	HIGH=An LOW=Aus
D3	GPIO0	Blau	Motor A Richtung	HIGH=Links LOW=Rechts

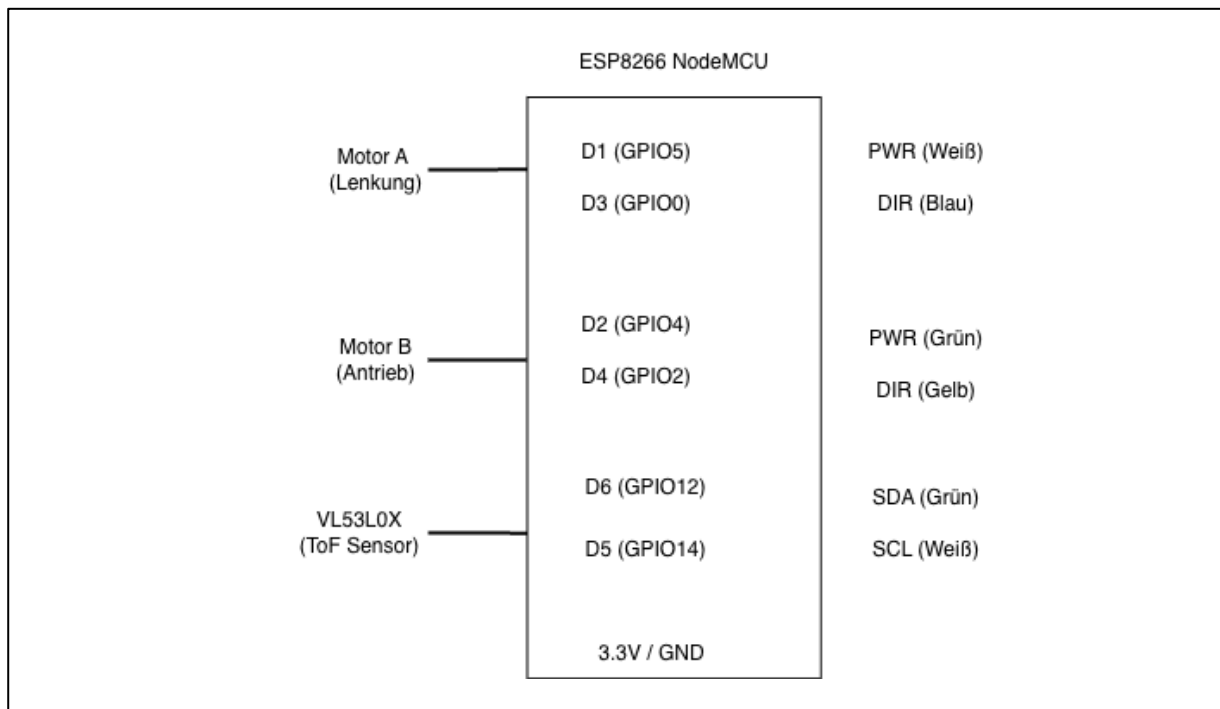
Motor B (Antrieb):

Pin	GPIO	Farbe	Funktion	Logik
D2	GPIO4	Grün	Motor B Ein/Aus	HIGH=An LOW=Aus
D4	GPIO2	Gelb	Motor B Richtung	HIGH=Rückwärts LOW=Vorwärts

VL53L0X Sensor (I2C)

Pin	GPIO	Farbe	Funktion
D6	GPIO12	Grün	SDA (Daten)
D5	GPIO14	Weiß	SCL (Serial Clock)

Die folgende Darstellung visualisiert die Zuordnung der Pins am ESP8266 und verdeutlicht die Verbindung zwischen Mikrocontroller und den angeschlossenen Komponenten.



Software-Komponenten

In diesem Abschnitt werden die zentralen Softwarekomponenten des Backends beschrieben, die für die Steuerung des Fahrzeugs sowie die Verarbeitung der eingehenden Befehle verantwortlich sind.

Zustandsvariablen

Zur Umsetzung der Steuerung werden verschiedene Zustandsvariablen verwendet, die das aktuelle Verhalten des Fahrzeugs definieren.

Für die grundlegende Fahrzeugbewegung werden folgende Zustände verwaltet:

- `currentDirection`: beschreibt die aktuelle Bewegungsrichtung des Fahrzeugs (Stillstand: 0, Vorwärts: 1 oder Rückwärts: 2)
- `currentTurn`: definiert die Lenkbewegung (geradeaus: 0, links: 1 oder rechts: 2)

Darüber hinaus werden Zustände für den autonomen Modus berücksichtigt:

- currentMode: bestimmt, ob das Fahrzeug manuell oder autonom betrieben wird
- turnBool: unterstützt die interne Entscheidungslogik im autonomen Fahrmodus

Zusätzlich wird ein Zeitparameter (**driveTime**) verwendet, der die Dauer einzelner Bewegungsbefehle steuert und somit das Bewegungsverhalten des Fahrzeugs beeinflusst.

Hauptkomponenten

In diesem Abschnitt werden die zentralen Komponenten der Steuerungslogik des Backends beschrieben.

Eine wesentliche Komponente ist die Verarbeitung eingehender Befehle, die über die Kommunikationsschnittstelle empfangen werden. Diese Befehle werden interpretiert und in entsprechende Zustandsänderungen des Fahrzeugs überführt.

Die folgende Tabelle zeigt die unterstützten Steuerbefehle und deren Wirkung auf das System:

Befehl	Auswirkung
forward	Vorwärtsfahren (currentDirection = 1)
backward	Rückwärtsfahren (currentDirection = 2)
left	Links lenken (currentTurn = 1)
right	Rechts lenken (currentTurn = 2)
stop	Anhalten (currentDirection = 0, currentTurn = 0)
auto	Autonomen Modus aktivieren (currentMode = 1)
autostop	Autonomen Modus deaktivieren (currentMode = 0)

Eine weitere zentrale Funktion übernimmt die Motorsteuerung. Hierbei werden die Zustände für Bewegung und Lenkung kombiniert und in konkrete Steuerimpulse für die Motoren umgesetzt. Dabei erfolgt eine Trennung zwischen Antrieb (Vorwärts- und Rückwärtsbewegung) und Lenkung (links, rechts, geradeaus).

Eine weitere zentrale Komponente ist die Sensorverarbeitung, die die Grundlage für das autonome Verhalten des Fahrzeugs bildet. Der eingesetzte Distanzsensor wird initial konfiguriert, wobei verschiedene Parameter zur Anpassung der Messgenauigkeit und Reichweite festgelegt werden. Dazu gehören unter anderem ein Long-Range-Modus zur Erhöhung der Messdistanz sowie ein angepasstes Timing-Budget für präzisere Messungen. Die Erfassung der Distanz erfolgt kontinuierlich über den Sensor und liefert Messwerte, die zur Hinderniserkennung genutzt werden. Im Falle eines Messfehlers wird ein entsprechender Rückgabewert erzeugt, wodurch eine robuste Verarbeitung der Sensordaten sichergestellt wird. Auf Basis dieser Messwerte wird im autonomen Fahrmodus eine Entscheidungslogik angewendet. Befindet sich ein Hindernis innerhalb eines definierten Abstandsbereichs, reagiert das System mit einer Anpassung der Bewegung. Liegt kein Hindernis im kritischen Bereich vor, setzt das Fahrzeug seine Vorwärtsbewegung fort.

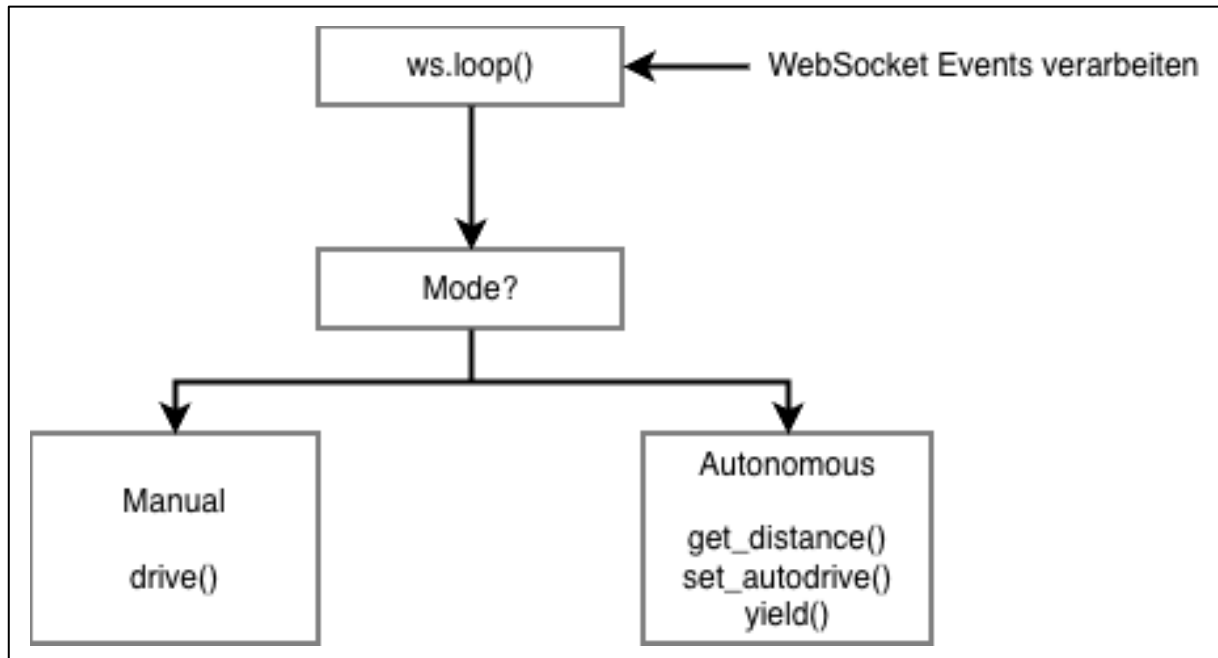
Programmablauf

Im Folgenden wird der grundlegende Ablauf des Programms auf dem Mikrocontroller beschrieben. Dieser besteht aus einer Initialisierungsphase sowie einer kontinuierlich ausgeführten Hauptschleife.

Zu Beginn wird das System initialisiert. Dabei werden alle notwendigen Komponenten vorbereitet, um den Betrieb des Fahrzeugs zu ermöglichen. Die Initialisierung umfasst insbesondere:

1. Serielle Kommunikation initialisieren
2. VL53L0X Sensor initialisieren
3. Motor-Pins als OUTPUT konfigurieren
4. WiFi Access Point starten
5. WebSocket-Server starten

Nach der Initialisierung wird die kontinuierliche Hauptschleife ausgeführt. In dieser werden eingehende Befehle verarbeitet, Sensordaten ausgelesen und die entsprechenden Steuerbefehle für die Motoren umgesetzt. Der Ablauf innerhalb der Hauptschleife ist im folgenden Diagramm dargestellt.



Kommunikationsprotokoll

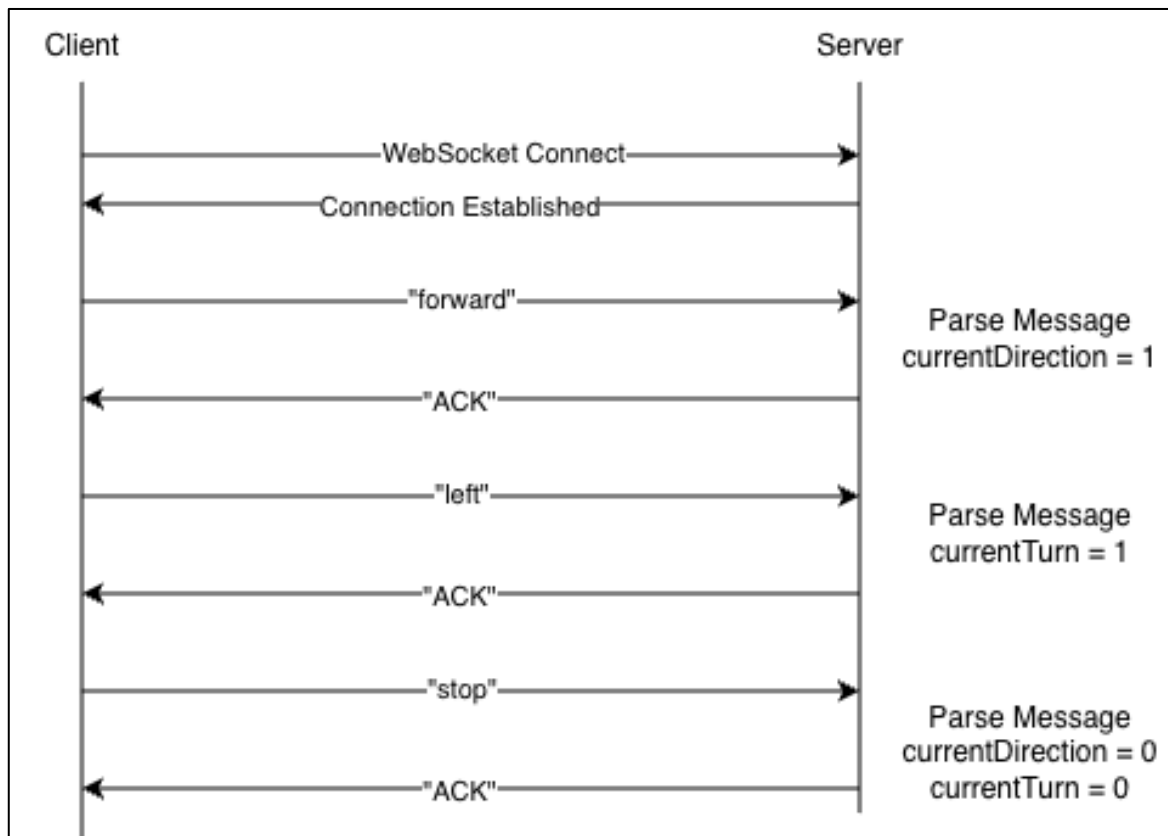
Für die Kommunikation zwischen Applikation und Mikrocontroller wird das WebSocket-Protokoll verwendet. Die Verbindung erfolgt über die URL „ws://10.10.10.10:81“.

Die Netzwerkparameter sind wie folgt definiert:

- SSID: R.E.D
- IP-Adresse: 10.10.10.10
- Subnetzmaske: 255.255.255.0
- Gateway: 10.10.10.10
- Modus: Access Point (AP)
- WebSocket-Port: 81

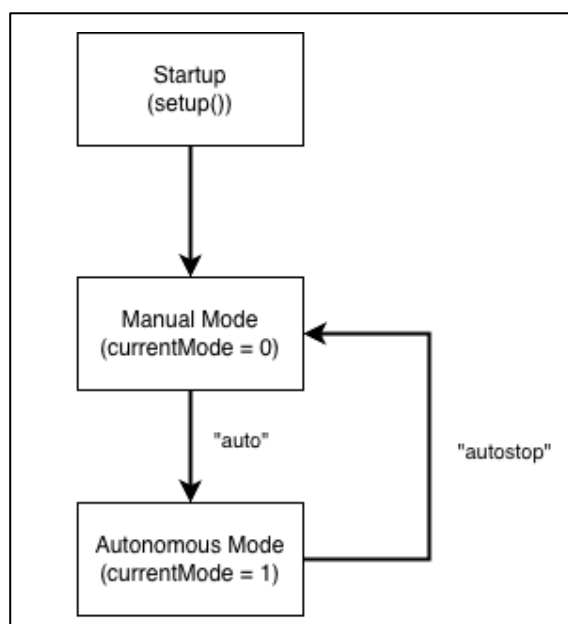
Diese Konfiguration ermöglicht eine direkte Verbindung zwischen Endgerät und Fahrzeug.

Der Ablauf der Kommunikation wird im folgenden Diagramm als Beispiel dargestellt.

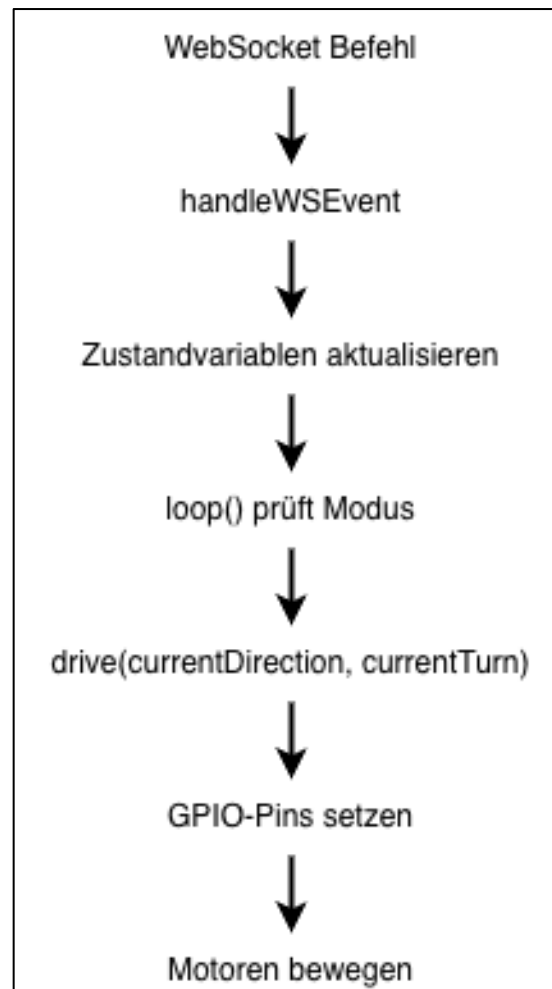


Betriebsmodi

Die Betriebsmodi definieren das Verhalten des Fahrzeugs und unterscheiden zwischen manueller und autonomer Steuerung. Das folgende Zustandsdiagramm veranschaulicht die möglichen Betriebszustände sowie die Übergänge zwischen diesen Modi.

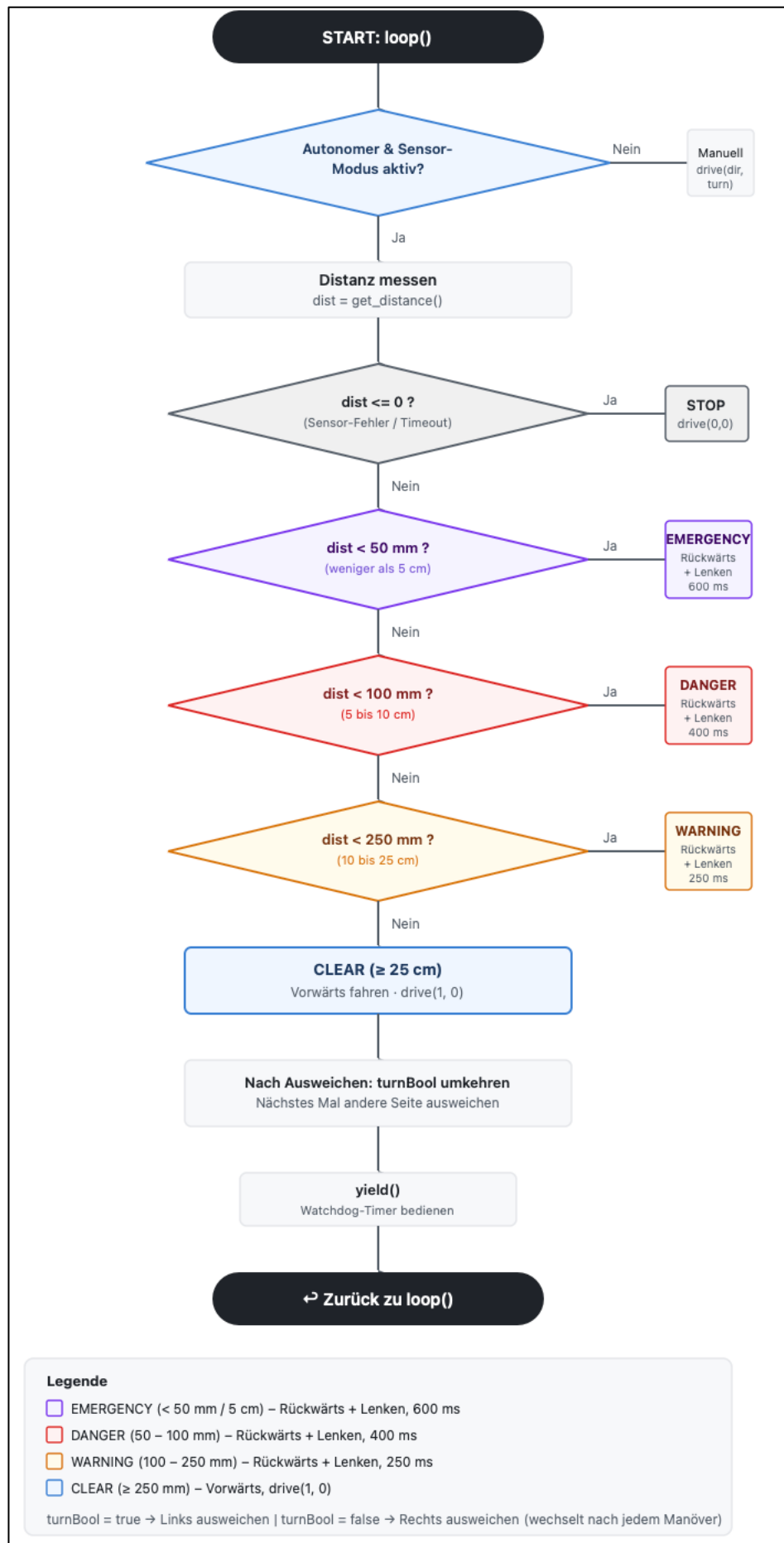


Standardmäßig befindet sich das System im manuellen Modus. In diesem Zustand wird das Fahrzeug direkt durch Eingaben des Nutzers gesteuert. Der Ablauf sowie der Datenfluss in diesem Modus werden in den entsprechenden Diagrammen dargestellt.



Zusätzlich kann der autonome Modus durch einen Steuerbefehl aus der Applikation aktiviert werden. In diesem Betriebszustand erfolgt die Steuerung des Fahrzeugs eigenständig auf Basis der Sensordaten, ohne direkte Nutzereingaben.

Der Datenfluss zwischen den einzelnen Komponenten während des autonomen Betriebs wird im anschließenden Diagramm visualisiert. Dabei wird insbesondere deutlich, wie Sensordaten, Steuerlogik und Motorsteuerung zusammenwirken.



Tests und Qualitätssicherung

Zur Sicherstellung der Funktionalität, Stabilität und Benutzerfreundlichkeit des Systems wurden im Rahmen des Projekts systematische Testfälle definiert und durchgeführt. Die Testkonzeption orientiert sich an grundlegenden Prinzipien des Software-Engineerings insbesondere an der strukturierten Testfallermittlung und -durchführung.

Ein wesentlicher Bestandteil der Teststrategie war die Verwendung von Äquivalenzklassen. Hierbei werden Eingaben und Systemzustände in Gruppen unterteilt, bei denen ein gleichartiges Systemverhalten erwartet wird.

Im Rahmen des Projektes wurden folgende Äquivalenzklassen identifiziert. Diese sind in der Tabelle dargestellt.

Kategorie	Äquivalenzklasse	Beschreibung
Verbindungszustände	Keine Verbindung	Keine Verbindung zum Fahrzeug möglich
	Erfolgreiche Verbindung	Stabile Verbindung mit WebSocket
	Instabile Verbindung	Wiederholte Verbindungsabbrüche
Eingabemethoden	Touch-Eingaben	Steuerung über Buttons im UI
	Tastatureingaben	Steuerung über WASD/Pfeiltasten
Fahrzeugzustände	Stillstand	Fahrzeug bewegt sich nicht
	Vorwärts- und Rückwärtsbewegungen	Lineare Bewegung
	Richtungsänderung	Lenkbewegung links/rechts

Nutzerinteraktionen	Kurze Eingabe	Tippen/Klicken
	Langes Halten	Gedrückt halten eines Buttons
	Mehrfacheingaben	Gleichzeitige Steuerbefehle
Routenverarbeitung	Gerade Strecken	Keine Richtungsänderung
	Leichte Kurven	Kleine Winkeländerungen
	Scharfe Kurven	Große Richtungsänderungen
	Komplexe Routen	Kombination mehrerer Segmente
Algorithmische Grenzfälle	Kurze Segmente	Minimale Streckenlänge
	Lange Segmente	Große Streckenlängen
	Kleine Winkel	Kaum erkennbare Richtungsänderung
	Große Winkel	Stark ausgeprägte Kurven
Systemverhalten und Performanceverhalten	Normale Nutzung	Standardnutzung
	Inaktive Verbindung	Keine Eingabe über längere Zeit
	Hohe Eingabefrequenz	Schnelle wiederholte Eingaben
Zustandsmanagement	Inkonsistenter Zustand	Veraltete Befehle sind aktiv
	Bereinigter Zustand	Nur die aktuellen Eingaben sind aktiv
Systeminitialisierung	Sensorprüfung nicht durchgeführt	Autonomer Modus ohne durchgeführte Sensorabfrage

	Sensorprüfung bestätigt	Sensor wurde durch den Nutzer bestätigt und aktiviert
	Sensorprüfung abgelehnt	Autonomer Modus ohne aktivierten Sensor
Befehlsverarbeitung	Sofortige Befehlsausführung	Befehle werden ohne Verzögerung an das Fahrzeug gesendet
	Verzögerte Befehlsausführung	Befehle werden zeitverzögert ausgeführt (z.B. durch Timer)
	Fehlende Befehlsausführung	Befehle werden nicht oder verspätet gesendet

Auf Basis dieser Äquivalenzklassen wurden konkrete Testfälle entwickelt, die typische sowie kritische Systemzustände gezielt abdecken. Dabei wurde für jede Äquivalenzklasse mindestens ein repräsentativer Testfall definiert, um das erwartete Systemverhalten valide zu überprüfen.

Die Durchführung der Tests erfolgte iterativ während des Entwicklungsprozesses. Dabei wurden Fehler systematisch identifiziert und analysiert. Für jeden Fehler wurden die Ursachen bestimmt, entsprechende Korrekturen implementiert und anschließend wiederholt getestet.

Die Testfälle umfassen in strukturierter Form:

- Beschreibung des Testziels
- Vorbedingungen
- Definierte Testschritte
- Erwartete Ergebnisse
- Tatsächliche Ergebnisse
- Analyse der Fehlerursache
- Implementierte Korrekturmaßnahmen

- Abschließende Verifikation

Die Ergebnisse zeigen, dass vor der Implementierung der Bugfixes mehrere Testfälle fehlschlagen, insbesondere in den Bereichen Verbindungsstabilität, Eingabeverarbeitung und Routenalgorithmik.

Nach der Umsetzung der identifizierten Verbesserungen konnten alle Testfälle erfolgreich bestanden werden.

Zu den wichtigsten Optimierungen zählen:

- Verbesserung der Verbindungsstabilität durch einen Keepalive-Mechanismus
- Korrekte Synchronisation von Benutzerinteraktionen und Systemzuständen
- Optimierung der Routenverarbeitung durch angepasste Parameter und Segmentierung
- Steigerung der Reaktionsfähigkeit der Steuerung durch Entfernung verzögernder Mechanismen

Die vollständige Testdokumentation ist in den folgenden ergänzenden Dokumenten enthalten:

- Test-Cases-Connection-Fixes
- Test-Cases-main-dart-Bugfixes

Diese enthalten die detaillierte Beschreibung aller Testfälle einschließlich Vorbedingungen, Testschritte, erwartete und tatsächliche Ergebnisse

Reflexion / Fazit

Zusammenfassend ist diese Dokumentation eine umfassende Darstellung des Projektverlaufs sowie der gewählten Vorgehensweise.

Die Entwicklung des Systems erfolgte strukturiert und auf Basis gemeinsam getroffener Architektur- und Funktionsentscheidungen. Diese wurden zunächst in Form von User Stories und User Cards definiert und anschließend in einer Anforderungsanalyse systematisch ausgearbeitet. Dabei wurde insbesondere darauf geachtet, zentrale Funktionalitäten frühzeitig zu implementieren, um eine stabile Grundlage für weiterführende Komponenten zu schaffen.

Der Großteil der Anforderungen konnte erfolgreich umgesetzt werden. Dazu zählen insbesondere die Integration und Verdrahtung der Hardware, die Kommunikation zwischen Applikation und Fahrzeug, die manuelle Steuerung, die Umsetzung verschiedener Betriebsmodi sowie die Berücksichtigung der Benutzerfreundlichkeit, beispielsweise durch den Rechts- und Linkshändermodus. Darüber hinaus wurden sowohl das Routenzeichnen als auch das autonome Fahren erfolgreich realisiert.

Zur Überprüfung der Funktionalität wurde die Applikation sowohl auf mobilen Endgeräten getestet als auch auf mehreren Fahrzeugen umgesetzt. Dies diente insbesondere der Demonstration der Übertragbarkeit und Reproduzierbarkeit des Systems. Dabei zeigte sich, dass sämtliche Kernfunktionen zuverlässig auf unterschiedlichen Fahrzeugen ausgeführt werden konnten, auch wenn nur eines der Fahrzeuge mit einem Sensor für den autonomen Betrieb ausgestattet war.

Im Verlauf des Projekts zeigte sich, dass das autonome Fahren eine zentrale Grundlage für das semi-autonome Fahren darstellt, da grundlegende Funktionen wie das Stoppen und Ausweichen bereits darin umgesetzt wurden. Aus diesem Grund wurde die Priorisierung der User Stories im Projektverlauf angepasst und das autonome Fahren vorgezogen. Diese Anpassungen verdeutlichen ein agiles Vorgehen, bei dem Anforderungen kontinuierlich überprüft und auf Basis neuer Erkenntnisse angepasst wurden.

Darüber hinaus konnte im Bereich der Kartenfunktion nicht die vollständige geplante User Story umgesetzt werden. Während das Zeichnen einer Route erfolgreich

realisiert wurde, konnten das automatische Scannen der Umgebung sowie die Erklärung der Funktionsweisen aus zeitlichen Gründen nicht vollständig implementiert werden. Auch das ursprünglich geplante semi-autonome Fahren konnte innerhalb des Projektrahmens nicht vollständig umgesetzt werden.

Eine weitere Limitierung ergab sich durch die verwendete Hardware. Der eingesetzte Sensor zeigte in der Praxis eine eingeschränkte Erkennungsfähigkeit, insbesondere bei dunklen oder nicht reflektierenden Oberflächen. Dadurch war die Hinderniserkennung in bestimmten Umgebungen nicht durchgängig zuverlässig, was die Robustheit des autonomen Fahrverhaltens beeinflusste.

Trotz der zuvor geschilderten Problematiken konnte ein funktionsfähiges Gesamtsystem entwickelt werden, das grundlegende Konzepte des Software-Engineerings erfolgreich vereint. Hierzu zählen insbesondere die modulare Architektur, das agile Vorgehen, eine strukturierte Anforderungsanalyse sowie systematische Testverfahren.

Für zukünftige Weiterentwicklungen bieten sich insbesondere die automatische Umgebungserkennung sowie eine Erweiterung der Hinderniserkennung im Fahrmodus und die Erklärung der Funktionsweisen in der Applikation an. Darüber hinaus könnte durch den Einsatz verbesserter Sensorik die Zuverlässigkeit erhöht werden. Auch eine Absicherung der Kommunikation durch geeignete Verschlüsselungsverfahren stellt eine mögliche Erweiterung dar.

Insgesamt zeigt das Projekt eine gelungene Verbindung von Hard- und Softwarekomponenten zur praxisnahen Demonstration zentraler Konzepte des Software-Engineerings.